

BudgetMem: 面向运行时智能体记忆的查询感知预算分层路由

Haozhen Zhang^{*1} Haodong Yue^{*2} Tao Feng³ Quanyu Long¹ Jianzhu Bao¹ Bowen Jin³
Weizhi Zhang⁴ Xiao Li⁵ Jiaxuan You³ Chengwei Qin⁶ Wenya Wang¹

摘要

对于需要跨越单个上下文窗口运行的大语言模型 (LLM) 智能体, 记忆正变得愈发重要。然而, 现有系统大多依赖离线、查询无关的记忆构建, 这种方式不仅效率不高, 还可能丢弃对当前查询至关重要的信息。运行时利用记忆虽然是一种自然的替代方案, 但既有工作往往会引入可观的额外开销, 并且难以显式控制性能与成本之间的权衡。本文提出 **BudgetMem**, 一种面向运行时智能体记忆的框架, 用于实现显式、查询感知的性能-成本控制。BudgetMem 将记忆处理组织为一组记忆模块, 每个模块均提供三个预算档位, 即 低/中/高。轻量级路由器在各模块间执行预算档位路由, 以平衡任务性能与记忆构建成本; 该路由器由一个通过强化学习训练的紧凑神经策略实现。以 BudgetMem 为统一试验平台, 我们研究了实现预算档位的三种互补策略: 实现分层 (方法复杂度)、推理分层 (推理行为) 和容量分层 (模块模型规模)。在 LoCoMo、LongMemEval 和 HotpotQA 上, 当系统优先追求性能 (即高预算设置) 时, BudgetMem 超越了强基线; 在预算更紧张时, 它也能取得更优的准确率-成本前沿。此外, 我们的分析厘清了不同分层策略各

自的优势与局限, 说明在不同预算区间内, 哪一种维度能够带来最有利的权衡。代码已发布于 <https://github.com/ViktorAxelsen/BudgetMem>。

1. 引言

记忆已成为现代大语言模型 (LLM) 智能体的核心组件, 使其能够超越单个上下文窗口保留并复用信息, 从而支持长程交互、个性化与知识密集型推理 (Hu et al., 2025)。然而, 既有工作大多聚焦于离线、查询无关的记忆构建: 它们以固定方式预处理、压缩或索引历史上下文, 而不考虑下游查询 (Kang et al., 2025; Fang et al., 2025; Zhong et al., 2024; Xu et al., 2025; Packer et al., 2023; Chhikara et al., 2025)。这种“一次构建, 始终使用”的范式既可能浪费资源, 也较为脆弱: 无论具体查询需要什么, 它都会消耗计算, 同时还可能遗漏某些查询所需的关键信息。

一种直观的替代方案是按需记忆抽取, 即根据当前查询在运行时触发计算。这种灵活性也有代价: 记忆处理被推迟到运行时, 使成本与延迟成为首要问题。在实际应用中, 工业级 LLM 系统越来越多地提供显式且通常分档的计算控制, 例如“思考”模式 (Singh et al., 2025)、推理等级 (OpenAI, 2025) 或更重型的模型选项 (Anthropic, 2025); 这反映出系统需要在质量与运行时成本之间取得平衡。由此, 智能体记忆面临一个关键问题: 如何为运行时记忆抽取提供显式且可控的性能-成本权衡?

遗憾的是, 在运行时智能体记忆中实现性能-成本权衡存在根本性困难。现有权衡机制大多在离线阶段工作, 而按需记忆会把这些决策推迟到运行时, 使

^{*} 同等贡献, 联系方式: haozhen001@e.ntu.edu.sg¹ 南洋理工大学² 清华大学³ 伊利诺伊大学厄巴纳-香槟分校⁴ 伊利诺伊大学芝加哥分校⁵ 中山大学⁶ 香港科技大学 (广州)。通讯作者: Chengwei Qin <chengweiqin@hkust-gz.edu.cn>。

每个查询都需要在质量与成本之间作出选择。这暴露出两个核心问题。第一，预算应施加在哪里？现有系统常将记忆视为采用固定计算设置的单体流水线，导致权衡粒度粗糙且难以控制 (Kang et al., 2025; Zhong et al., 2024)。因此，需要为运行时场景定义合适的预算单元，即确定记忆抽取过程中的哪些模块应被分配预算，从而有针对性且有效地控制计算。第二，预算应如何实现？既有工作很少为运行时记忆的权衡提供系统性指导，往往诉诸临时性的成本削减，或仅仅增加计算量 (Yan et al., 2025a)。因此，即使预算方案已经确定，如何落实预算控制、哪些设计维度最能表达有意义的权衡，以及这些选择在不同预算区间下如何表现，仍不清楚。要回答这些问题，就必须超越一次性的启发式方法或高计算量的简单升级，以更系统的视角研究运行时智能体记忆的性能-成本控制。

为解决上述挑战，我们提出 **BudgetMem**，一个运行时智能体记忆框架，通过按需记忆抽取实现显式、可控的性能-成本权衡。BudgetMem 将运行时记忆抽取视为多阶段模块化流水线，并使计算能够在模块级得到控制。具体而言，BudgetMem 通过统一的预算档位接口规范各模块的调用方式，使学习得到的路由器可以在保持整体抽取结构不变的同时，为每个模块选择不同预算档位。

在这一模块化骨架之上，BudgetMem 为每个模块提供三个预算档位，即低/中/高，对应不同的质量-成本权衡。我们通过三种互补的分层策略实现预算档位：实现分层（改变模块实现）、推理分层（改变推理行为）和容量分层（改变模块的模型容量）。为在这些分层选项中进行选择，BudgetMem 使用一个共享的轻量级路由器，在处理查询的过程中执行预算档位路由：路由器在每个模块处根据可用上下文，即查询与模块中间状态，选择一个档位。我们采用带成本感知奖励的强化学习来训练路由器，在任务性能与记忆抽取成本之间进行权衡，从而形成可控的性能-成本行为。这一设计既得到了一套实用、可控的运行时记忆系统，也支持在统一框架内比较不同的预算实现策略。

在 LoCoMo、LongMemEval 和 HotpotQA 上的实验表明，在性能优先的高预算设置下，BudgetMem 相比有竞争力的基线取得显著提升；随着预算收紧，它也呈现出清晰的性能-成本权衡曲线。此外，我们的分析分离了不同预算分层策略的相对优势，为判断不同预算区间内哪种机制能带来最佳收益提供了依据。

本文贡献总结如下：

- 我们提出 **BudgetMem**，一种模块化运行时智能体记忆框架，通过具有预算档位的模块，对按需记忆抽取实施显式的性能-成本控制。
- 我们提出预算档位路由，使用强化学习训练一个共享的轻量级路由器，在抽取过程中选择预算档位；并在统一框架内进一步实现三种互补的预算策略，即实现分层、推理分层和容量分层。
- LoCoMo、LongMemEval 和 HotpotQA 上的实验展示了强劲性能与清晰的性能-成本权衡；进一步分析还揭示了不同策略在各预算区间内何时能够取得最佳收益。

2. 相关工作

2.1. 记忆增强 LLM 智能体

记忆增强 LLM 智能体通常维护一个外部记忆库，以突破有限上下文窗口的限制并支持长期使用。大量既有工作强调离线或预先构建记忆：系统定期对历史交互进行摘要或压缩并建立索引，随后在查询时通过检索访问这些内容 (Zhong et al., 2024; Packer et al., 2023; Lee et al., 2024; Kang et al., 2025; Fang et al., 2025)。代表性设计按照时间顺序和层次结构组织记忆，例如事件摘要与用户画像，并结合信息检索和基于时近性的衰减等启发式更新规则 (Zhong et al., 2024; Kang et al., 2025)。

较新的工作通过智能体式更新和结构化设计丰富记忆。例如，有方法构建包含丰富元数据的笔记，并将其连接成图，以支持可扩展的检索与演化 (Xu

et al., 2025); 还有方法使用基于 LLM 的记忆管理器检索相似条目并执行离散操作, 即添加、更新、删除和不操作, 其变体还会构建知识图谱等结构化记忆 (Chhikara et al., 2025)。一些方法进一步引入基于学习的记忆管理, 通常采用强化学习, 并利用下游任务信号优化记忆操作 (Yan et al., 2025b; Wang et al., 2025)。按需利用方法也不再局限于“先检索、再回答”, 而是会在记忆之上执行更深入的规划 (Yan et al., 2025a)。

尽管取得了这些进展, 运行时记忆的成本依然高昂, 而且鲜少被表述为显式的性能-成本控制问题: 大多数工作依赖固定流水线, 或主要在离线构建阶段研究效率 (Fang et al., 2025)。与之不同, 我们聚焦运行时记忆抽取, 并系统比较预算约束下可控的性能-成本权衡。

2.2. LLM 系统中的推理时性能-成本权衡

越来越多的研究通过提供可控的“计算旋钮”, 探索如何在 LLM 质量与运行时成本、延迟之间进行权衡。现有方法大体分为三类。(i) 算法与系统级优化在不改变任务的情况下削减推理成本, 包括更快的解码 (Fu et al., 2024; Cai et al., 2024; Li et al., 2024b)、提前退出或自适应深度推理 (Schuster et al., 2022)、剪枝与稀疏化 (Ma et al., 2023; Frantar & Alistarh, 2023; Sun et al., 2023)、量化 (Xiao et al., 2023a; Liu et al., 2024), 以及缓存、批处理和面向长上下文的 KV 缓存效率等服务端优化 (Zhang et al., 2023; Xiao et al., 2023b; Ge et al., 2023; Li et al., 2024a)。(ii) 推理级控制根据不同预算改变推理行为, 例如直接生成与思维链 (Wei et al., 2022)、自我精炼和反思循环 (Yao et al., 2022; Shinn et al., 2023; Madaan et al., 2023), 以及通过限制步骤数、采样数或搜索范围实现有界思考 (Wang et al., 2022)。(iii) 容量级控制改变有效模型容量, 包括混合专家激活 (Shazeer et al., 2017; Fedus et al., 2022) 与基于蒸馏的部署 (Gu et al., 2023; Agarwal et al., 2024); 这与预算约束下跨后端的 LLM 路由密切相关 (Chen et al., 2024; Feng et al., 2024; Zhang et al., 2025; Jin et al., 2025)。

这些方向虽然为可控权衡提供了实用机制, 但在运

行时智能体记忆中尚未得到系统研究; 在该场景下, 计算开销用于记忆抽取, 而不是答案生成。本文迈出第一步, 通过引入预算分层框架, 并在统一设置下研究互补的预算实现策略, 把显式的性能-成本控制带入运行时智能体记忆。

3. 问题设定与方法概览

本文研究运行时智能体记忆抽取: 系统在查询时选择性处理原始历史记录, 构建紧凑记忆, 以回答当前用户查询。与预先压缩或预先结构化全部历史的离线流水线不同, 我们保持历史记录原样不变, 并将记忆计算推迟到查询到达之后, 从而避免查询无关的预处理所造成的不可逆信息损失。在详述方法之前, 我们先从输入和输出两个角度简要概括任务设定。

输入。 令 $\mathcal{H}$ 表示 LLM 智能体的历史, 例如先前的对话、日志或文档。我们首先执行轻量级、任务无关的切分, 把 $\mathcal{H}$ 划分为分块库 $\mathcal{C} = \{c_i\}_{i=1}^N$, 其中每个 c_i 都是一个短文本块¹。给定用户查询 q , 检索器 $\mathcal{R}$ 返回一个可能相关的文本块子集:

$$\mathcal{C}_q = \mathcal{R}(q, \mathcal{C}), \quad \mathcal{C}_q \subset \mathcal{C}. \quad (1)$$

其中, $(q, \mathcal{C}_q)$ 作为运行时记忆流水线的输入。

输出。 系统生成: (i) 紧凑且对查询有用的抽取记忆 m ; (ii) 由 LLM 在 (q, m) 条件下生成的最终答案 $\hat{y}$:

$$m = f_{\text{mem}}(q, \mathcal{C}_q), \quad \hat{y} = f_{\text{ans}}(q, m). \quad (2)$$

这里, f_{mem} 是学习得到、受预算控制的记忆抽取过程, 也是本文的研究重点; f_{ans} 则是答案生成器, 例如一个固定的 LLM。

带预算的运行时抽取。 一个关键挑战在于, 运行时抽取在线发生且成本可能很高; 因此, 我们希望实现显式且可控的性能-成本权衡。为此, 我们设计了一条模块化记忆流水线, 其中每个模块均提供一小

¹该步骤仅切分原始文本以便建立索引, 不执行任何离线抽取、摘要或改写。

组预算档位（低/中/高）。在处理查询的过程中，共享的轻量级路由器作出预算档位路由决策，根据可用上下文，即查询与中间信号，为每个模块选择所用档位。最终模块生成的抽取记忆随后用于回答查询；端到端过程产生任务级性能信号与抽取成本信号，用于以端到端方式优化路由器。

4. BudgetMem

本节首先介绍 BudgetMem 的模块化流水线骨架（第 4.1 节），随后形式化预算档位并给出三种分层策略（第 4.2 节），最后说明如何利用强化学习优化轻量级路由器，学习预算档位路由（第 4.3 节）。

4.1. 模块化运行时记忆流水线

如图 1 所示，BudgetMem 将 f_{mem} 实现为一条多阶段模块化流水线，逐步把检索到的原始文本块精炼为聚焦当前查询的记忆。针对以对话为中心的评估，我们采用一个简单、可解释的模块组合，即过滤 $\rightarrow$ 并行抽取 $\rightarrow$ 摘要，将其作为研究预算控制的具体骨架。需要说明的是，实验虽采用了这一实例，框架本身并不假定特定的模块集合，也可以适配其他模块化记忆流水线。

流水线结构。 给定 (q, C_q) ，BudgetMem 执行一条固定的模块化流水线：

$$\mathcal{M}_{\text{fil}} \rightarrow \{\mathcal{M}_{\text{ent}}, \mathcal{M}_{\text{tmp}}, \mathcal{M}_{\text{top}}\} \rightarrow \mathcal{M}_{\text{sum}}. \quad (3)$$

从整体上看， $\mathcal{M}_{\text{fil}}$ （过滤模块）对检索到的上下文进行精炼；并行模块 $\{\mathcal{M}_{\text{ent}}, \mathcal{M}_{\text{tmp}}, \mathcal{M}_{\text{top}}\}$ （实体、时间和主题模块）抽取互补的中间上下文； $\mathcal{M}_{\text{sum}}$ 则把它们汇总为最终记忆。前序模块的输出会被送入后续模块，形成由流水线逐阶段精炼的中间状态。本文关注预算档位控制与路由，而非具体模块实现，因此将模块细节放在附录 A.5。

模块接口与中间状态。 我们使用包含查询与当前可用信号的上下文来表示每次模块调用。过滤模块生成经过精炼的文本块集合：

$$\tilde{C}_q = \mathcal{M}_{\text{fil}}(q, C_q). \quad (4)$$

其中， $\tilde{C}_q$ 可视为一个更聚焦的文本块子集或经过重新加权的文本块集合。接着，三个抽取模块在过滤后的上下文上并行运行：

$$\begin{aligned} e &= \mathcal{M}_{\text{ent}}(q, \tilde{C}_q), & t &= \mathcal{M}_{\text{tmp}}(q, \tilde{C}_q), \\ p &= \mathcal{M}_{\text{top}}(q, \tilde{C}_q). \end{aligned} \quad (5)$$

这里， e, t, p 表示抽取出的上下文，例如以实体为中心的事实、时间线索和主题摘要；它们可以表示为文本、结构化字段或紧凑的自然语言笔记。最后，摘要模块把这些输出汇总为抽取记忆：

$$m = \mathcal{M}_{\text{sum}}(q, e, t, p), \quad (6)$$

然后将其提供给答案生成器，即 $\hat{y} = f_{\text{ans}}(q, m)$ 。

预算档位接口（概念）。 流水线中的每个模块 $\mathcal{M}$ 都通过统一的预算档位接口对外提供能力，这意味着模块可以在不同计算预算（低/中/高）下被调用，同时保持相同的输入-输出契约。这一标准化设计支持模块级预算控制，并为学习预算档位路由（第 4.3 节）和比较不同分层策略（第 4.2 节）奠定基础。

4.2. 预算档位与分层策略

BudgetMem 为流水线中的每个模块 $\mathcal{M}$ 配置一小组预算档位，从而在运行时抽取期间显式提供质量-成本选项。具体而言，每个模块都提供低/中/高三个档位，分别对应逐渐增加的计算预算以及通常更高的抽取质量。重要的是，预算档位通过统一的预算档位接口（第 4.1 节）在各模块内部定义，使路由器无须改变流水线结构，便能逐模块作出细粒度预算决策。

分层策略。 一个关键问题是如何在实践中实现这些档位。受现代 LLM 系统常用性能-成本控制旋钮的启发（第 2.2 节），我们研究三种互补的分层策略，分别捕捉不同的成本变化来源：(i) **实现分层**，改变模块实现，从轻量级启发式方法，到学习得到的任务专用模型，再到基于 LLM 的处理；(ii) **推理分层**，在保持底层模型骨架不变的情况下改变推理行为，例如直接生成与更审慎的推理模式；(iii) **容量分层**，改变模块内部使用的模型容量，例如使用

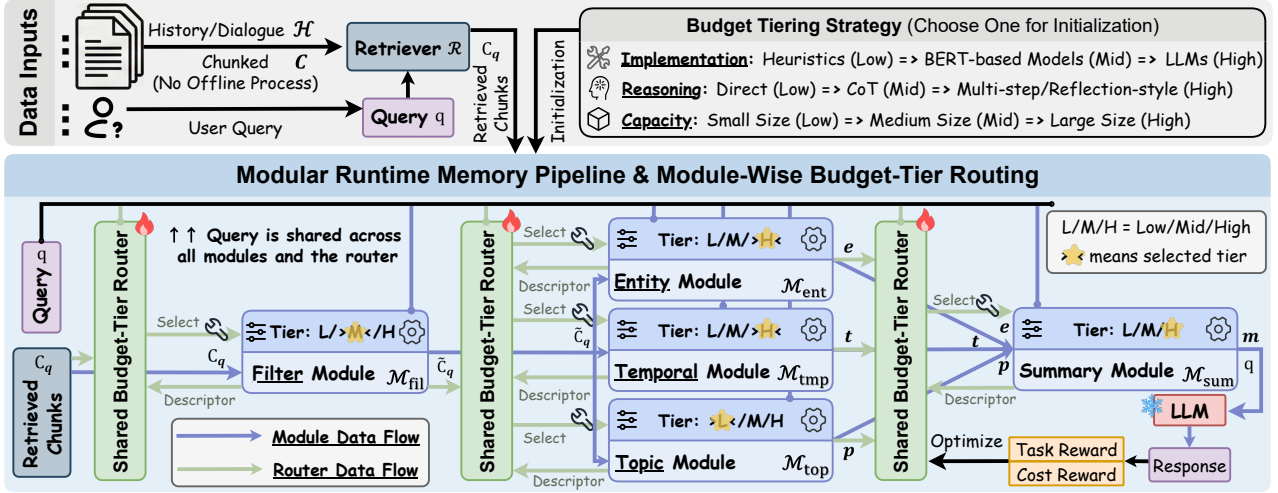


图 1. BudgetMem 概览。给定用户查询 q ，系统从经过分块的历史中检索原始文本块 C_q （不执行离线记忆预处理），并通过模块化流水线进行处理（过滤 → 实体/时间/主题 → 摘要）。每个模块提供低/中/高预算档位，并采用三种策略之一实现，即**实现分层**、**推理分层**或**容量分层**。共享的轻量级路由器根据查询与中间状态为每个模块选择档位，并使用任务奖励与成本奖励进行强化学习训练，从而得到可控的性能-成本权衡。（ C_q ：检索到的原始文本块； $\tilde{C}_q$ ：过滤后的文本块； e, t, p ：抽取的上下文； m ：抽取记忆）

较小或较大的语言模型。这些策略彼此正交：它们通过不同机制，即算法过程、推理模式和模型规模，交换计算资源，因此可以在统一框架内直接比较各自的权衡特性。

低/中/高档位的实现。 对于**实现分层**，低档采用轻量级规则或模式处理，中档采用针对模块功能训练的紧凑学习模型，通常为基于 BERT 的模型 (Devlin et al., 2019)，高档则升级为基于 LLM 的处理，以提高抽取质量。对于**推理分层**，各档位通过计算量逐步增加的推理行为实现，例如低档为直接生成，中档为思维链式推理 (Wei et al., 2022)，高档为多步或反思式推理 (Shinn et al., 2023)。对于**容量分层**，各档位对应实现同一模块功能时采用的不同模型规模。附录 A.5 给出了每种策略在各模块上的具体实例。

4.3. 学习模块级预算档位路由

给定分层模块，BudgetMem 学习一个共享的轻量级路由器，在整个模块化流水线中执行预算档位路由。随着查询被逐阶段处理，路由器为每次模块调用选择预算档位，从而针对单个查询产生一系列档位决策。由于记忆抽取过程可能包含不可微组件，我们

将路由表述为序列决策问题，并使用强化学习优化路由器。

策略、状态与动作。 令 π_θ 表示参数为 θ 的路由策略。在每个模块调用步骤 k ，路由器观测概括可用上下文的状态 s_k ，并输出动作 $a_k \in \{\text{低, 中, 高}\}$ ，为当前模块选择档位。具体而言，我们使用以下信息构造 s_k ：(i) 查询 q ；(ii) 当前模块输入，即上一模块的输出；(iii) 指示当前被路由模块的模块描述符²。所选动作 a_k 决定模块执行时使用的预算档位，模块随后生成下一组中间信号，进而形成下一个状态。

回合与目标。 针对一个查询完整运行一次模块化流水线构成一个回合。执行所有经过路由的模块后，我们得到抽取记忆 m ，并生成答案 $\hat{y} = f_{\text{ans}}(q, m)$ ，据此计算任务性能奖励 $r_{\text{task}} \in [0, 1]$ 。此外，经过路由的模块执行会产生抽取成本，我们将其转换为成本奖励 r_{cost} 。我们在一个成本感知目标下优化路由

²我们把状态实现为这些组成部分的紧凑嵌入；细节见附录 A.4。

器，以权衡任务性能与抽取成本：

$$r = r_{\text{task}} + \lambda \cdot \alpha \cdot r_{\text{cost}}, \quad (7)$$

其中， λ 控制性能-成本权衡偏好， α 是用于对齐奖励量级的缩放因子，具体如下所述。我们使用标准策略优化算法，例如 PPO (Schulman et al., 2017)，来优化 π_θ ；算法细节见附录 A.4。

成本建模。 我们把原始抽取成本定义为经过路由的流水线上各模块成本之和：

$$c_{\text{raw}} = \sum_k c(\mathcal{M}_k, a_k). \quad (8)$$

对于基于 LLM 的档位，我们使用词元用量乘以相应的输入/输出词元价格来度量 $c(\cdot)$ ；相较之下，非 LLM 档位，例如轻量级启发式方法，其成本被视为可忽略不计。为使成本项与 r_{task} 具有可比尺度，我们采用滑动窗口归一化，将成本映射到有界范围 (Zhang et al., 2025)。具体而言，我们维护一个近期原始成本窗口，并使用稳健分位数对 $\sqrt{c_{\text{raw}}}$ 进行归一化：

$$\tilde{c} = \frac{\sqrt{c_{\text{raw}}} - Q_5}{Q_{95} - Q_5}, \quad r_{\text{cost}} = 1 - \text{clip}(\tilde{c}, 0, 1), \quad (9)$$

其中， Q_5 和 Q_{95} 表示在滑动窗口上计算的第 5 和第 95 百分位数。

奖励尺度对齐。 在实践中，我们观察到 r_{task} 和 r_{cost} 在训练期间可能呈现不同的方差，这会使得方差较大的项主导策略更新。为缓解这一问题，我们引入一个简单的、基于方差的对齐因子：

$$\alpha = \frac{\text{std}(r_{\text{task}})}{\text{std}(r_{\text{cost}}) + \epsilon}, \quad (10)$$

其中， $\text{std}(\cdot)$ 在近期训练奖励上计算， ϵ 为一个较小常数。式 7 中的最终奖励因而同时兼顾：(i) 通过 λ 实现可控权衡；(ii) 通过 α 稳定优化。

5. 实验设置

5.1. 数据集、指标与基线

数据集。 我们在三个强调长程记忆与长上下文问答 (QA) 的基准上评估 BudgetMem：广泛用于智

能体记忆评估的 LoCoMo (Maharana et al., 2024) 和 LongMemEval (Wu et al., 2024)，以及作为代表性长上下文问答任务的 HotpotQA (Yang et al., 2018)³。

指标。 我们同时报告任务性能与成本，以刻画性能-成本权衡。任务性能方面，我们使用 F1 分数 (F1) 与 LLM 评审 (Judge)，衡量模型预测相对于真实答案的正确性。成本方面，我们汇总 BudgetMem 在抽取期间所有模型调用的 API 用量（输入和输出词元），并依据相应服务价格把词元数量换算为货币成本，从而度量记忆抽取成本⁴。

基线。 我们将 BudgetMem 与一组多样且强大的记忆增强基线进行比较：(1) ReadAgent (Lee et al., 2024)；(2) MemoryBank (Zhong et al., 2024)；(3) A-MEM (Xu et al., 2025)；(4) LangMem (LangChain, 2025)；(5) Mem0 (Chhikara et al., 2025)；(6) MemoryOS (Kang et al., 2025)；(7) LightMem (Fang et al., 2025)。这些基线覆盖了广泛的记忆架构，因而能够与既有智能体记忆系统进行全面比较。更多细节见附录 A。

5.2. 实现细节

我们使用 LLaMA-3.3-70B-Instruct (Grattafiori et al., 2024) 和 Qwen3-Next-80B-A3B-Instruct (Yang et al., 2025) 作为基础 LLM，并通过 API 服务访问。我们以 LLaMA 作为记忆抽取骨架来训练预算档位路由器，随后不经重新训练，直接使用同一个已训练路由器，以 Qwen 为骨架进行测试。对于检索和分块，我们把每段历史或文档切分为固定长度的文本块 (LoCoMo 和 LongMemEval 的默认块大小为 256，HotpotQA 为 1024)，并使用 Contriever (Izacard et al., 2021) 作为默认检索器。针对每个查询，我们先检索一个候选文本块集合，然后对下游记忆处理统一采用相同的 top- K 预算 ($K=5$)，使 BudgetMem 与所有基线在相同检索上下文大小下运行。

训练方面，我们在各数据集上采用 6/2/2 的训练/验

³我们遵循既有工作 (Yu et al., 2025) 采用的评估协议。

⁴<https://www.together.ai/pricing>

表 1. 性能优先设置下 LoCoMo、LongMemEval 和 HotpotQA 的实验结果, 指标包括 F1 分数 (F1)、LLM 评审 (Judge) 和成本。

方法	LoCoMo			LongMemEval			HotpotQA			平均		
	F1	评审	成本 [↓]	F1	评审	成本 [↓]	F1	评审	成本 [↓]	F1	评审	成本 [↓]
LLaMA-3.3-70B-Instruct												
ReadAgent	22.48	31.05	0.57	20.75	27.72	13.68	15.33	30.08	4.19	19.52	29.62	6.14
MemoryBank	22.27	28.98	0.73	26.74	32.67	3.94	22.25	23.75	7.75	23.75	28.47	4.14
A-MEM	26.43	32.96	2.88	21.74	33.17	80.02	43.25	54.69	26.74	30.47	40.27	32.07
LangMem	22.31	25.96	0.48	12.00	17.00	16.60	22.78	22.66	10.95	19.03	21.87	9.34
Mem0	11.04	28.18	2.89	27.70	42.08	13.57	28.03	36.72	4.30	22.26	35.66	6.92
MemoryOS	30.62	34.55	1.97	12.97	33.50	38.83	34.50	43.36	13.32	26.03	37.14	18.04
LightMem	33.88	40.76	1.50	26.74	48.51	5.28	45.73	58.37	10.10	35.45	49.21	5.63
BudgetMem-IMP	38.75	50.32	1.80	37.47	56.00	0.71	49.31	65.77	1.35	41.84	57.36	1.29
BudgetMem-REA	40.92	52.23	2.90	40.53	58.00	0.67	51.12	61.93	0.99	44.19	57.39	1.52
BudgetMem-CAP	43.05	54.62	2.40	40.24	60.50	0.80	53.87	64.85	0.93	45.72	59.99	1.38
Qwen3-Next-80B-A3B-Instruct[†]												
ReadAgent	22.45	31.37	0.24	27.72	20.75	4.97	18.05	25.78	1.75	22.74	25.97	2.32
MemoryBank	23.53	34.71	0.25	10.56	28.22	3.45	18.64	31.25	1.79	17.58	31.39	1.83
A-MEM	27.65	38.54	2.88	10.82	31.19	21.00	40.54	50.39	8.34	26.34	40.04	10.74
LangMem	20.89	23.40	0.14	11.01	14.00	3.99	20.77	21.09	17.56	17.56	19.50	2.82
Mem0	10.77	25.32	1.15	25.71	36.14	4.96	24.72	37.89	2.02	20.40	33.12	2.71
MemoryOS	35.43	38.85	0.75	13.35	33.00	15.84	41.21	53.52	11.68	30.00	41.79	9.42
LightMem	32.85	42.83	0.70	27.70	47.52	3.39	41.29	55.42	8.56	33.95	48.59	4.21
BudgetMem-IMP	40.14	54.38	0.80	29.18	52.00	0.30	46.67	57.42	0.63	38.66	54.60	0.58
BudgetMem-REA	40.19	53.34	1.11	35.84	59.00	0.26	57.67	70.83	0.17	44.57	61.06	0.51
BudgetMem-CAP	41.22	53.18	0.61	31.01	56.00	0.17	58.70	72.08	0.22	43.64	60.42	0.33

粗体表示每个基础模型分组中的最佳分数;“平均”为各指标在不同数据集上的均值。

成本通过汇总所有模型调用的词元, 并采用相应模型的输入与输出词元价格计算。

[†] 表示未使用该基础模型训练, 仅进行迁移评估。

证/测试划分, 并对所有方法保持相同的数据划分与评估协议, 以确保公平比较。我们采用近端策略优化 (PPO) (Schulman et al., 2017) 作为默认强化学习算法, 并使用 Adam 优化器、32 的批大小, 最多训练 600 步。附录 A 和附录 C 给出了其他超参数、提示词与实现细节。

6. 实验分析

实验组织如下。第 6.1 节报告性能优先设置 ($\lambda = 0$) 的结果, 并比较三个变体: **BudgetMem-IMP**、**BudgetMem-REA** 和 **BudgetMem-CAP**。第 6.2 节通过改变 λ 描绘性能-成本曲线; 第 6.3 节消融成本建

模与奖励设计; 第 6.4 节给出进一步分析。

6.1. 主要结果

如表 1 所示, 我们在性能优先设置下, 于 LoCoMo、LongMemEval 和 HotpotQA 上评估 BudgetMem, 并与多种代表性记忆系统进行比较。结果支持以下三个关键发现。

有效性。 BudgetMem 在三个数据集上都取得显著提升, 并在 F1 和 LLM 评审两项指标上持续优于既有方法。例如, 在采用 LLaMA-3.3-70B 骨架的 LongMemEval 上, BudgetMem-CAP 的评审分数达到 60.50, 大幅超过最强基线 LightMem 的 48.51, 说明



图 2. LoCoMo 上不同分层策略的性能-成本权衡。通过改变成本权重 λ , BudgetMem 描绘出平滑、可控的前沿; 随着预算增加, 该前沿向更高性能移动, 并在低成本和高成本区间均包络基线。

其利用长上下文证据的能力更强, 答案质量也更高。

在可控成本下取得强劲性能。即使在性能优先区间, BudgetMem 仍具有较高成本效率, 能够在不产生过量开销的情况下提升质量。尤其是在使用 Qwen3-Next-80B-A3B 的 HotpotQA 上, BudgetMem-CAP 以 0.22 的成本取得最佳评审分数 72.08; BudgetMem-REA 则以更低的 0.17 成本取得接近的 70.83。这种效率源于 BudgetMem 的运行时按需设计: 它不在离线阶段处理全部历史, 而是检索与查询相关的原始文本块, 并且只在需要时投入抽取计算。LoCoMo 是一个例外: 该数据集的历史较短, 离线流水线产生的开销也较小, 因此不同方法之间的成本差异不那么明显。

汇总结果上的整体性能最强。汇总各数据集的结果后, BudgetMem 各变体在每个骨架模型分组中都稳定名列前茅, 说明其整体有效性很强, 增益并非局限于单个基准。该趋势在 LLaMA 和 Qwen 上均成立, 表明在性能优先区间内, BudgetMem 能在多样的评估设置中普遍取得强劲表现。

6.2. 探索不同分层策略的权衡

如图 2 所示, 我们系统比较了 LoCoMo 上三个分层维度的性能-成本分布。随着预算约束放宽, 即 λ 减小, BudgetMem 在成本-性能平面上呈现平滑的左上移动趋势, 形成连续且可控的权衡前沿。该前沿在低成本与高成本区间均包络既有基线, 能够在相近成本下取得更高评审分数, 或在相近性能下实现

更低成本。

值得注意的是, 实现分层与容量分层覆盖了更宽的成本范围: 实现分层在中等预算下迅速提高评审分数; 随着预算增加, 容量分层继续把前沿向外推进, 并在高预算区间取得最佳质量。相比之下, 推理分层呈现明显的成本集中现象。其主要调整推理行为, 例如直接生成与反思, 同时保持底层模型容量不变, 因此额外成本主要来自相对稳定的词元开销, 成本分布范围更窄。这一模式表明, 推理维度更像是在有限成本带宽内工作的细粒度质量旋钮: 它能以相近成本带来有意义的改进, 但相比实现分层和容量分层, 不太适合探索极低预算设置, 也难以外推至极高预算性能。总体而言, 从预算受限到性能优先区间, BudgetMem 始终推动帕累托前沿向外移动。不同分层维度之间的差异进一步表明, 实现分层和容量分层更适合扩大预算覆盖范围并外推性能边界, 而推理分层最适合在成本相对集中的区域内精细提升质量。

6.3. 奖励尺度对齐消融

如图 3 所示, 我们在 LoCoMo 上采用容量分层策略, 对奖励尺度对齐 (第 4.3 节) 进行消融。任务奖励与成本奖励在尺度和波动性上可能存在显著差异, 因此移除对齐会使优化不稳定, 并可能使学习偏向成本项, 最终得到退化的低成本策略。实验上, 在不进行奖励尺度对齐且成本权重 $\lambda=0.3$ 时, 路由器坍塌为对大多数模块选择低档, 答案质量显著下降, 评

审分数也降至最低。相比之下, 启用奖励尺度对齐后, 路由器学会更有层次地使用不同档位, 产生平滑得多、行为也更合理的性能-成本前沿。该消融说明, 奖励尺度对齐对于平衡学习信号十分重要; 它使系统实现有意义的成本-性能控制, 而不是退化为简单的成本最小化。



图 3. 奖励尺度对齐消融。在 LoCoMo 上采用容量分层策略。

6.4. 讨论

预算档位选择比例。 图 4 采用容量分层策略, 通过报告不同成本权重 λ 下低/中/高档位的选择比例, 分析 LongMemEval 上的模块级路由行为。总体而言, 路由器呈现出清晰、可解释的预算响应: 随着成本压力增大, 它系统地把概率质量从低成本档位转移至更便宜的档位, 从模块层面证明 BudgetMem 会以成本感知方式分配计算。

更具体地说, 当 λ 较小, 例如 0.1 时, 路由器把大多数模块分配到 中档, 优先保证质量。当 $\lambda = 0.3$ 时, 它在保留可观 中档比例的同时提高 低档占比, 体现出对计算量的可控削减。在更大的 λ 下, 为满足更严格的成本偏好, 策略进一步在各模块上集中选择低档。总体来看, 这些趋势证实 BudgetMem 会随着预算偏好变化, 以可预测的方式调节不同档位的使用。

检索规模敏感性分析。 图 5 展示了在三种分层策略下, 改变 LoCoMo 上检索原始文本块数量时的成本和评审分数。由于输入更长且需要额外处理, 增加检索规模会如预期般提高成本; 更多支持证据也往往会提高评审分数, 这体现了证据覆盖范围与计算开销之间的标准权衡。

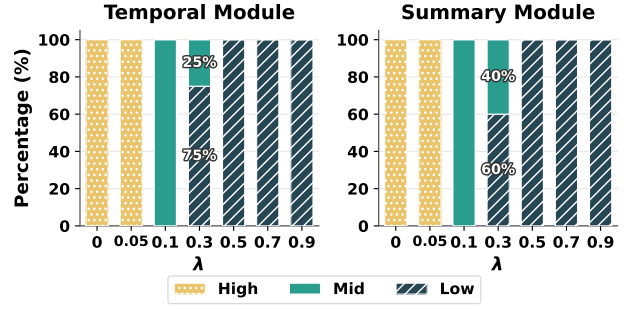


图 4. 预算档位选择比例。采用容量分层策略时, LongMemEval 上不同成本权重 λ 所对应的模块级低/中/高路由比例。

然而, 收益并非单调增长。在我们的设置中, 检索 5 个文本块能在成本与质量之间取得最佳平衡。检索过多文本块可能降低性能: 额外文本块会引入更多冗余或弱相关内容, 增加噪声并可能分散 LLM 的注意力, 从而降低评审分数。反之, 检索块过少会导致证据不足, 限制下游增益。总体而言, 这些结果表明, 检索规模是一个重要的实用控制旋钮, 可在证据充分性、上下文噪声与成本之间取得平衡。

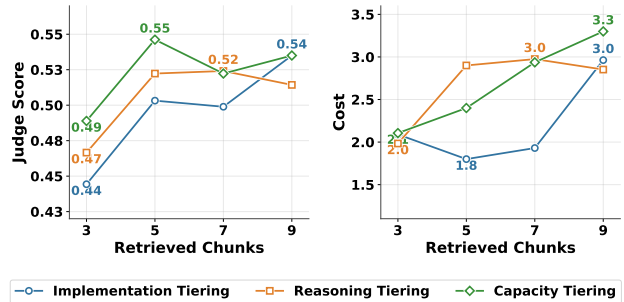


图 5. LoCoMo 上的检索规模敏感性。在三种分层策略下, 成本和评审分数随检索原始文本块数量的变化。

延迟分析。 尽管我们的主要成本指标聚焦货币成本, 延迟对于实际部署同样重要。因此, 我们在受控的本地部署中额外测量延迟: 所有方法共享 Qwen 骨架, 并采用相同的硬件与推理栈。该比较不使用 API 延迟, 因为网络状况与服务提供方调度会对其产生显著影响, 不适合进行公平的相对比较。表 2 和表 3 报告该设置下的绝对延迟; 我们主要利用它们比较不同方法的相对流水线开销。

如表 2 所示, 在不同预算设置下, BudgetMem 呈现

出预期的延迟权衡。对于实现分层,提高成本权重会把总推理延迟从 $\lambda = 0$ 时的 3881 毫秒降至 $\lambda = 0.9$ 时的 1167 毫秒,同时检索/过滤延迟从 1176 毫秒降至 46 毫秒。推理分层与容量分层也会降低延迟,但降幅较小,因为其档位主要改变推理行为或模型容量,而不是替换过滤实现。此外,测得的 GPU 时间始终较短,每个查询约为 44–52 毫秒,说明轻量级路由器自身引入的开销有限。如表 3 所示,所有方法均能从批处理中受益,且 BudgetMem 的相对开销在不同服务设置下大体一致。总体而言,这些结果说明检索/过滤开销在实践中是可控的;对完整流水线进行系统级加速与本文目标正交,留待未来工作研究。

表 2. 受控本地部署下 LoCoMo 的详细延迟。批大小为 1,所有数值的单位均为毫秒。离线表示平均离线记忆构建时间,总计表示平均总推理延迟,过滤表示平均检索/过滤模块延迟, GPU 表示平均 GPU 时间。N/A 表示不适用或未测量。

方法	离线	总计	过滤	GPU
MemoryOS	40657	2842	1488	N/A
A-MEM	26842	1449	49	N/A
LightMem	9740	1662	62	N/A
本文-IMP ($\lambda = 0$)	N/A	3881	1176	46
本文-IMP ($\lambda = 0.3$)	N/A	2432	556	46
本文-IMP ($\lambda = 0.9$)	N/A	1167	46	46
本文-REA ($\lambda = 0$)	N/A	3678	1274	47
本文-REA ($\lambda = 0.3$)	N/A	3429	1216	45
本文-REA ($\lambda = 0.9$)	N/A	3318	1209	48
本文-CAP ($\lambda = 0$)	N/A	3461	1228	44
本文-CAP ($\lambda = 0.3$)	N/A	3058	1079	46
本文-CAP ($\lambda = 0.9$)	N/A	2853	976	52

表 3. LoCoMo 上的批大小扩展。我们报告不同批大小下的平均总推理延迟,所有数值单位均为毫秒。BS 表示批大小。BudgetMem 报告 $\lambda = 0$ 的本文-REA。

BS	MemoryOS	A-MEM	LightMem	本文-REA
1	2842	1449	1662	3678
2	1633	845	1059	2295
4	974	518	775	1327
8	604	296	438	858
16	387	154	269	547
32	203	96	163	326

7. 结论

本文提出 BudgetMem, 一个运行时智能体记忆框架,用于在按需记忆抽取中实现显式的性能-成本控制。BudgetMem 为模块化记忆流水线中的每个模块配置低/中/高预算档位,并学习一个轻量级路由器,在成本感知目标下选择档位。以 BudgetMem 为统一试验平台,我们比较了三种互补的分层策略,即实现分层、推理分层与容量分层,并刻画它们在不同预算下的权衡行为。LoCoMo、LongMemEval 和 HotpotQA 上的实验表明,该方法在性能优先设置下具有强劲表现,并能在预算收紧时改善性能-成本前沿;实验还揭示了各策略分别在何种条件下带来最佳收益。

致谢

本研究/项目得到新加坡南洋理工大学启动基金(#023284-00001)、新加坡教育部学术研究基金一级种子项目(RS37/24, #025041-00001)以及广东省科协青年科技人才培养计划(SKXRC2025462)的支持。

影响声明

BudgetMem 通过使记忆计算在预算约束下可被显式控制,解决记忆增强智能体在实际部署中的问题。这有助于那些要求延迟或成本可预测的应用,也能降低资源受限场景使用智能体记忆的门槛。

BudgetMem 是一种架构与训练框架,不会引入记忆增强智能体既有数据来源之外的新数据。因此,其潜在风险与现有记忆系统相似,例如偶尔检索到无关上下文,或暴露无害但并非有意呈现的历史细节。在实践中,合理的保留策略、基本访问控制以及对失败案例的常规评估等标准防护措施可以缓解这些问题。总体而言,我们期望 BudgetMem 的主要影响是在现实预算约束下提升智能体记忆的可用性与可控性。

参考文献

- Agarwal, R., Vieillard, N., Zhou, Y., Stanczyk, P., Garea, S. R., Geist, M., and Bachem, O. On-policy distillation of language models: Learning from self-generated mistakes. In *The twelfth international conference on learning representations*, 2024.
- Anthropic. Claude code. <https://github.com/anthropics/claude-code>, 2025. GitHub repository.
- Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.
- Chen, S., Jiang, W., Lin, B., Kwok, J., and Zhang, Y. Routerdc: Query-based router by dual contrastive learning for assembling large language models. *Advances in Neural Information Processing Systems*, 37: 66305–66328, 2024.
- Chhikara, P., Khant, D., Aryan, S., Singh, T., and Yadav, D. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186, 2019.
- Fang, J., Deng, X., Xu, H., Jiang, Z., Tang, Y., Xu, Z., Deng, S., Yao, Y., Wang, M., Qiao, S., et al. Lightmem: Lightweight and efficient memory-augmented generation. *arXiv preprint arXiv:2510.18866*, 2025.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Feng, T., Shen, Y., and You, J. Graphrouter: A graph-based router for llm selections. *arXiv preprint arXiv:2410.03834*, 2024.
- Frantar, E. and Alistarh, D. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International conference on machine learning*, pp. 10323–10337. PMLR, 2023.
- Fu, Y., Bailis, P., Stoica, I., and Zhang, H. Break the sequential dependency of llm inference using lookahead decoding. *arXiv preprint arXiv:2402.02057*, 2024.
- Ge, S., Zhang, Y., Liu, L., Zhang, M., Han, J., and Gao, J. Model tells you what to discard: Adaptive kv cache compression for llms. *arXiv preprint arXiv:2310.01801*, 2023.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Gu, Y., Dong, L., Wei, F., and Huang, M. Minillm: Knowledge distillation of large language models. *arXiv preprint arXiv:2306.08543*, 2023.
- Hu, Y., Liu, S., Yue, Y., Zhang, G., Liu, B., Zhu, F., Lin, J., Guo, H., Dou, S., Xi, Z., et al. Memory in the age of ai agents. *arXiv preprint arXiv:2512.13564*, 2025.
- Izacard, G., Caron, M., Hosseini, L., Riedel, S., Bojanowski, P., Joulin, A., and Grave, E. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118*, 2021.
- Jin, B., Collins, T., Yu, D., Cemri, M., Zhang, S., Li, M., Tang, J., Qin, T., Xu, Z., Lu, J., et al. Controlling performance and budget of a centralized multi-agent llm system with reinforcement learning. *arXiv preprint arXiv:2511.02755*, 2025.

- Kang, J., Ji, M., Zhao, Z., and Bai, T. Memory os of ai agent. *arXiv preprint arXiv:2506.06326*, 2025.
- LangChain. Langmem. <https://github.com/langchain-ai/langmem>, 2025. GitHub repository.
- Lee, K.-H., Chen, X., Furuta, H., Canny, J., and Fischer, I. A human-inspired reading agent with gist memory of very long contexts. *arXiv preprint arXiv:2402.09727*, 2024.
- Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., and Chen, D. Snapkv: Llm knows what you are looking for before generation. *Advances in Neural Information Processing Systems*, 37: 22947–22970, 2024a.
- Li, Y., Wei, F., Zhang, C., and Zhang, H. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024b.
- Liu, Z., Oguz, B., Zhao, C., Chang, E., Stock, P., Mehdad, Y., Shi, Y., Krishnamoorthi, R., and Chandra, V. Llm-qat: Data-free quantization aware training for large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 467–484, 2024.
- Ma, X., Fang, G., and Wang, X. Llm-pruner: On the structural pruning of large language models. *Advances in neural information processing systems*, 36:21702–21720, 2023.
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhume, S., Yang, Y., et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36:46534–46594, 2023.
- Maharana, A., Lee, D.-H., Tulyakov, S., Bansal, M., Barbieri, F., and Fang, Y. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753*, 2024.
- OpenAI. Codex. <https://github.com/openai/codex>, 2025. GitHub repository.
- Packer, C., Fang, V., Patil, S., Lin, K., Wooders, S., and Gonzalez, J. Memgpt: Towards llms as operating systems. 2023.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Schuster, T., Fisch, A., Gupta, J., Dehghani, M., Bahri, D., Tran, V., Tay, Y., and Metzler, D. Confident adaptive language modeling. *Advances in Neural Information Processing Systems*, 35:17456–17472, 2022.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.
- Singh, A., Fry, A., Perelman, A., Tart, A., Ganesh, A., El-Kishky, A., McLaughlin, A., Low, A., Ostrow, A., Ananthram, A., et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.
- Sun, M., Liu, Z., Bair, A., and Kolter, J. Z. A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695*, 2023.
- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- Wang, Y., Takanobu, R., Liang, Z., Mao, Y., Hu, Y., McAuley, J., and Wu, X. Mem- $\{\alpha\}$: Learning memory construction via reinforcement learning. *arXiv preprint arXiv:2509.25911*, 2025.

- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K.-W., and Yu, D. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.
- Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International conference on machine learning*, pp. 38087–38099. PMLR, 2023a.
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023b.
- Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., and Zhang, Y. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.
- Yan, B., Li, C., Qian, H., Lu, S., and Liu, Z. General agentic memory via deep research. *arXiv preprint arXiv:2511.18423*, 2025a.
- Yan, S., Yang, X., Huang, Z., Nie, E., Ding, Z., Li, Z., Ma, X., Kersting, K., Pan, J. Z., Schütze, H., et al. Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*, 2025b.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W., Salakhutdinov, R., and Manning, C. D. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pp. 2369–2380, 2018.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- Yu, H., Chen, T., Feng, J., Chen, J., Dai, W., Yu, Q., Zhang, Y.-Q., Ma, W.-Y., Liu, J., Wang, M., et al. Memagent: Reshaping long-context llm with multi-conv rl-based memory agent. *arXiv preprint arXiv:2507.02259*, 2025.
- Zhang, H., Feng, T., and You, J. Router-r1: Teaching llms multi-round routing and aggregation via reinforcement learning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36:34661–34710, 2023.
- Zhong, W., Guo, L., Gao, Q., Ye, H., and Wang, Y. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19724–19731, 2024.

A. 更多实现细节

A.1. 数据集统计

表 4 汇总了实验使用的数据集统计信息，包括 LoCoMo (Maharana et al., 2024)、LongMemEval (Wu et al., 2024) 和 HotpotQA (Yang et al., 2018) 的训练/验证/测试划分大小（训练查询数量）以及平均上下文长度（词元数）。

对于 HotpotQA，我们遵循既有工作 (Yu et al., 2025) 的评估协议与数据集构建方式：拼接证据文档与干扰文档，形成长上下文，并使用该设置公开的相同预处理测试查询。

表 4. 数据集统计。

数据集	# 训练/验证/测试	平均上下文长度
LoCoMo (Maharana et al., 2024)	1,236/446/304	18.0K 词元
LongMemEval (Wu et al., 2024)	282/94/94	122.1K 词元
HotpotQA (Yang et al., 2018; Yu et al., 2025)	5,250/1,750/128	26.0K 词元

A.2. 评估细节

表 5 和表 6 报告评估配置。表 5 列出所有数据集共享的超参数，表 6 进一步给出与数据集相关的评估设置。

除非另有说明，我们使用三个不同的随机种子重复每项 BudgetMem 实验，并报告平均结果。这可以减弱随机初始化与训练随机性的影响，特别是对于基于强化学习的路由器。所有运行均采用相同的数据划分、检索设置和评估协议，以确保比较公平且可复现。

A.3. 路由器状态表示

为提高可复现性，我们补充说明第 4.3 节使用的路由器状态表示。对于每次模块调用，路由器状态由三个文本部分构成：用户查询、当前模块输入，以及指示当前被路由模块的模块描述符。我们分别使用 `all-mpnet-base-v2` 对这三个部分进行编码。由此，查询、模块输入和模块描述符各自得到一个 768 维嵌入。

随后，我们拼接查询嵌入与模块输入嵌入，并通过线性投影得到 256 维上下文表示。模块描述符嵌入则单独投影为 256 维模块表示。最后，我们把 256 维上下文表示与 256 维模块表示拼接为最终的 512 维路由器状态向量。该状态向量被送入共享的预算档位路由器，以预测当前模块在低、中和高三个档位上的选择分布。训练期间保持嵌入模型冻结，仅使用强化学习优化投影层与路由策略网络。

A.4. PPO 训练目标细节

我们使用近端策略优化 (PPO) 训练共享预算档位路由策略 π_θ ，使其沿固定的模块化运行时记忆流水线 $M_{\text{fil}} \rightarrow \{M_{\text{ent}}, M_{\text{tmp}}, M_{\text{top}}\} \rightarrow M_{\text{sum}}$ 作出查询感知、模块级的档位选择。

回合、动作与状态。 单个查询构成一个 PPO 回合：给定 (q, C_q) ，路由器依次为每次模块调用选择预算档位，执行经过路由的流水线以得到抽取记忆 m ，随后生成最终答案 $\hat{y} = f_{\text{ans}}(q, m)$ 。在每个路由步骤 k ，动作 a_k 选择模块档位。每个模块使用三元动作空间 {低, 中, 高}。状态 s_k 是以下信息的紧凑嵌入：(i) 查询

表 5. 超参数设置 (所有数据集共享)

超参数	取值	超参数	取值
LLM 解码温度	0.0	批大小	32
嵌入维度 (查询/记忆/描述符)	768	检索 Top- K	5
LLM 评审模型	openai/ gpt-oss-120b		
容量分层模块档位 (Llama 的低/中/高)	L: meta/llama-3.2-3b-instruct; M: meta/llama-3.1-8b-instruct; H: meta/llama-3.3-70b-instruct		
容量分层模块档位 (Qwen 的低/中/高)	L: qwen/qwen2.5-7b-instruct; M: qwen/qwq-32b; H: qwen/qwen3-next-80b-a3b-instruct		

表 6. 评估超参数 (数据集特定)。

超参数	LoCoMo	LongMemEval	HotpotQA
分块/块大小	256	256	1024
LLM 模型最大词元数	32	512	512

q ; (ii) 当前模块输入, 即上一阶段的中间输出; (iii) 指示当前被路由模块的模块描述符。所有模块共享一个统一的演员-评论家网络; 模块身份通过模块描述符嵌入注入, 从而在共享参数的同时保留模块特定的路由行为。

奖励: 经过归一化与对齐的性能-成本权衡。 完成一个查询的全部模块路由执行后, 我们根据答案质量 (F1 或 LLM 评审) 计算任务奖励 $r_{\text{task}} \in [0, 1]$, 并根据路由模块调用产生的抽取成本计算成本奖励 r_{cost} 。我们优化如下成本感知标量奖励:

$$r = r_{\text{task}} + \lambda \cdot \alpha \cdot r_{\text{cost}}. \quad (11)$$

其中, $r_{\text{task}} \in [0, 1]$ 是根据答案质量 (F1 或 LLM 评审) 计算的任务奖励; r_{cost} 是从路由模块调用产生的抽取成本推导出的归一化成本奖励; λ 控制性能-成本权衡; α 为奖励尺度对齐因子。

我们对原始成本应用滑动窗口归一化, 并将其映射为有界奖励 (式 9), 从而计算 r_{cost} ; 同时通过基于方差的奖励尺度对齐 (式 10) 计算 α 。

采用联合 (多模块) 似然比的 PPO 目标。 令 $\log \pi_{\theta}(a_k | s_k)$ 表示当前策略下逐模块的对数概率。由于每个查询回合会产生一个模块动作序列, 我们使用经过路由的轨迹上的联合动作概率:

$$\log \pi_{\theta}(\mathbf{a} | \mathbf{s}) = \sum_k \log \pi_{\theta}(a_k | s_k), \quad \rho = \exp(\log \pi_{\theta}^{\text{new}} - \log \pi_{\theta}^{\text{old}}). \quad (12)$$

其中, 在路由步骤 k , s_k 为路由器状态嵌入, $a_k \in \{\text{低, 中, 高}\}$ 为当前模块所选的预算档位动作; $\pi_{\theta}(\cdot)$ 是参数为 θ 的路由策略, $\log \pi_{\theta}(a_k | s_k)$ 是对应的对数概率; $\mathbf{s} = \{s_k\}_k$ 和 $\mathbf{a} = \{a_k\}_k$ 表示一个查询回合内完整的状态-动作序列, 其联合对数似然为 $\log \pi_{\theta}(\mathbf{a} | \mathbf{s})$; ρ 为更新策略 (上标 new) 与采集轨迹所用行为策略之间的 PPO 似然比。

随后, 我们使用比率 ρ 应用标准 PPO 截断替代目标, 并加入价值函数损失与熵奖励:

$$L = L_{\text{policy}} + c_v L_{\text{value}} - c_e H. \quad (13)$$

其中, L_{policy} 为 PPO 截断替代策略损失, L_{value} 为价值函数回归损失, H 为策略熵, c_v 和 c_e 分别是价值损失与熵奖励的系数。

我们使用单步蒙特卡洛回报估计优势; 经过路由的流水线完成后, 回合即终止, 并使用评论家作为基线。为保持稳定, 我们在各模块决策之间取熵的平均值而非求和, 防止熵项随路由模块数量扩大。

优化与超参数。 路由器训练默认使用 PPO 作为强化学习优化器。除非另有说明, 我们使用 Adam, 学习率为 3×10^{-4} , 截断参数 $\epsilon = 0.2$, 价值损失系数 $c_v = 0.5$, 熵系数 $c_e = 0.01$, 每次更新执行 4 个 PPO 轮次, 并把梯度范数截断在 0.5。训练批大小为 32, 最多训练 600 步。路由器使用 768 维嵌入 (查询/记忆/描述符), 与共享超参数设置一致。

A.5. 模块实现细节

统一预算档位接口。 我们采用统一的预算档位接口: 每个模块提供三个档位 (低/中/高)。预算档位只改变模块内部的计算量和推理能力, 使系统无须改变整体流水线结构便可逐模块路由。为控制不同档位间的提示词影响, 我们对提示方式进行标准化: 低/中/高档始终使用各模块的高档提示词模板。

A.5.1. 分层策略：实现分层

保持固定契约的预算档位。实现分层通过增加模块内部计算预算来实现 低/中/高档，同时保持其输入-输出契约不变。低档使用低成本符号流水线，例如规则、正则表达式和 spaCy 等轻量级 NLP，实现快速且可控的处理；中档升级为紧凑的学习型专用模型，通常采用 BERT 式编码器 (Devlin et al., 2019)，以中等成本改善语义建模；高档进一步升级为基于 LLM 的处理，以最高成本获得最强的开放式推理与跨上下文整合能力。

过滤模块。过滤模块通过单调升级内部相关性估计器实现预算分层：从低成本符号信号，逐步升级到语义表示，最终采用 LLM 评估。在低档，它使用稀疏词汇/模式匹配和轻量级启发式评分，速度快、确定性强，适合积极控制成本。在中档，它引入基于表示的相关性，例如嵌入与 BERT 式相似度，从而以中等计算预算找回超越精确重叠的语义匹配。在高档，它进一步使用 LLM 对候选项评分，使系统能在相关性取决于隐式指代、释义或多跳语义时进行上下文感知的辨别。重要的是，各档位均保持相同的输出契约，即排序后的 top- k 候选集合，因此改进可归因于内部计算增加，而非接口变化。

实体模块。实体关系模块采用相同的预算阶梯，但将其具体用于抽取结构化关系线索。在低档，它使用轻量级规则与浅层 NLP 线索，例如正则模式以及 spaCy 的命名实体识别/依存信号，以较高可控性抽取显式关系轨迹；但当关系为隐式或跨越多个文本块时，覆盖率有限。在中档，它使用紧凑的任务专用抽取器替代启发式方法，通常采用 BERT 式关系抽取，例如类似 REBEL 的三元组生成，从而以中等预算改善主语-关系-宾语三元组的归一化与覆盖率。在高档，它使用 LLM 执行开放域关系补全与抽象，以找回缺失连接，并把不同文本块中的同义关系整合为一致三元组；模块仍输出标准化关系列表，确保下游融合保持不变。

时间模块。时间关系模块按照“显式线索检测 → 学习型时间抽取 → 生成式时间推理”分层，使预算增加对应逐步增强的时间建模能力。在低档，它使用轻量级 NLP 识别显式时间表达并构建事件-时间锚点；当时间线索被直接陈述时，这种方式较为可靠。在中档，它采用紧凑的学习型抽取器，例如 BERT 式模型，选择与时间相关的关系并将其归一化为一致形式。在高档，它升级为基于 LLM 的时间推理，使系统能在时间线必须通过推断而非直接读取时，处理隐式顺序、跨文本块对齐与全局一致性检查。所有档位都保持相同输出契约，即标准化的时间事实/锚点集合。

主题模块。主题模块通过逐步增强话语级抽象来实现预算分层：从低成本词汇线索，升级到学习型表示，最终采用基于 LLM 的主题推理。在低档，它从关键词、类似 TF-IDF 的模式和简单转折标记等低成本词汇/统计线索中抽取可控的主题信号，以较低成本粗略描述主题内容及其转移。在中档，它加入紧凑的学习型表示，例如 BERT 式编码器和可由 NLP 结构辅助的轻量级分段/主题建模，在中等计算量下形成更稳定的主题单元和更可靠的转折检测。在高档，它使用 LLM 生成更高层次的主题关系与演化叙事，提高长上下文或异构上下文的抽象能力与可解释性。关键在于，每个档位仍输出相同的标准化主题线索列表，供下游摘要模块统一使用。

摘要模块。摘要模块通过改变证据融合深度实现预算分层，同时保持最终记忆格式不变。在低档，它生成确定性的证据提要，例如对检索记忆与线索进行简洁汇总/拼接，以尽量减少计算与失真。在中档，它输出结构化综合结果，把关键证据、实体、时间锚点和主题线索组织为可解析模板，无须调用昂贵的生成式推理便可提高可用性并降低脆弱性。在高档，它升级为基于 LLM 的整合式摘要，由模型融合多源线索、解决冲突并生成解释性的最终记忆表示，从而更好地支持复杂查询。各档位下的摘要均保持契约兼容，因此档位效果反映所分配的计算预算，而非下游预期的变化。

A.5.2. 分层策略：推理行为

通过行为分层推理实现带预算的推理。我们保持模块接口固定，通过逐步增强的推理行为为模块分配不同计算预算，以实现推理分层。具体而言，低档执行直接推理，中档执行思维链式推理，高档执行强调迭代精炼与全局一致性的多步/反思式推理。各档位下，每个模块都接收相同的条件输入，即查询和候选记忆或中间线索，并生成相同的标准化产物，例如排序后的子集或线索列表。因此，档位差异反映内部推理深度与词元用量的变化，而非流水线结构变化。

过滤模块。所有档位下，过滤模块都按相关性对检索文本块排序，并返回 $\text{top-}k$ 。低档直接评分，以最大化效率和确定性；中档使用思维链式评分，改善超越表面重叠的语义校准；高档采用多步/反思式评分，更好地处理相关性取决于隐式约束或跨记忆交互的情况。

实体模块。各档位均输出相同格式的关系列表，档位只影响关系抽取能力。低档通过直接推理抽取关系，优先保证显式陈述事实的精确率；中档使用思维链式推理，提高关系元组的覆盖率与归一化质量；高档使用多步/反思式推理，整合同义表述、协调相互竞争的候选关系，并确保不同记忆之间的关系一致性。

时间模块。时间模块始终输出相同的时间事实/锚点集合，档位仅改变其推断与归一化能力。低档通过直接推理识别显式锚点；中档使用思维链式推理，选择并归一化与查询相关的时间约束；高档使用多步/反思式推理，改善跨文本块对齐、推断隐式顺序并促进全局时间线一致性。

主题模块。所有档位都保持主题线索列表格式不变，同时逐步生成更高层次的主题抽象。低档通过直接推理生成粗粒度主题线索；中档使用思维链式推理，得到更稳定的主题关系与转移；高档使用多步/反思式推理，提高抽象质量，并在长且异构的上下文中维持全局连贯性。

摘要模块。所有档位都保持最终记忆接口不变，同时逐步改善证据汇总与融合。低档以最少推理执行直接综合；中档执行思维链式综合，把实体、时间和主题线索更好地组织为连贯结构；高档执行多步/反思式综合，整合多源证据、减少不一致，并提高复杂查询下的整体忠实度。

A.5.3. 分层策略：模块模型容量

对于**容量分层**，我们在保持模块接口与推理行为不变的情况下，替换同一模块内部使用的骨架 *LLM*，以落实低/中/高预算。为控制不同档位之间的提示词影响，我们进一步标准化提示方式：所有预算等级下的每个模块都采用同一个高档提示词模板。在实现中，各档位对应取自常用开放权重模型家族的递增容量：低档以小模型实例化模块，例如 Llama-3.2-3B-Instruct 和 qwen/qwen2.5-7b-instruct；中档采用中等模型，例如 Llama-3.1-8B-Instruct 和 qwen/qwq-32b；高档采用大模型，例如 Llama-3.3-70B-Instruct 和 qwen3-next-80b-a3b-instruct。

过滤模块。过滤模块在各档位保持相同的相关性评分和选择过程，仅改变评分骨架的容量。给定查询与候选记忆集合，它分配相关性分数，并为下游抽取返回排序后的 $\text{top-}k$ 子集。在低/中/高档下，该评分分别由逐渐增大的骨架执行，从而在相同 $\text{top-}k$ 契约下改善语义校准与稳健性。

实体模块。实体模块在各档位保持相同的抽取目标与输出格式，从过滤证据中生成标准化的实体中心关系列表。容量分层只影响用于实例化抽取器的模型；更大的骨架能更好地找回隐式关系，而输出的关系列表仍与下游融合接口兼容。

时间模块。时间模块在所有档位下执行固定的时间抽取过程，并输出标准化的时间事实/锚点集合。容量分

层只改变从过滤记忆中推断并归一化相关时间约束所用的底层骨架，在不修改模块契约的情况下，以运行时成本换取时间信息的忠实度与一致性。

主题模块。主题模块在所有档位下保持固定的主题关系抽取功能，并输出标准化主题线索列表。由于只改变骨架容量，更大的模型可以从相同证据中归纳出更稳定的主题结构与话语转移，而接口和下游使用方式保持不变。

摘要模块。摘要模块在各档位使用相同摘要接口，把上游模块输出汇总为最终的紧凑记忆表示。容量分层使用逐渐增大的骨架实例化摘要器，从而在复杂证据下提高综合稳健性与冲突处理能力，同时保持最终记忆格式与下游预期不变。

B. 更多实验结果

对于 LoCoMo (Maharana et al., 2024)，我们报告各类别的 F1 分数、评审分数与成本。结果针对数据集定义每个类别分别计算，以细粒度分析模型在不同长上下文对话场景中的表现。遵循既有做法，我们在评估中排除类别 5（对抗类）。为清楚比较不同模型家族，结果分别在两张表中报告，对应基于 LLaMA 和基于 Qwen 的模型。LoCoMo 各类别的详细结果见表 7 和表 8。

对于 LongMemEval (Wu et al., 2024)，我们给出包含 F1 分数、评审分数与成本的类别级评估结果。每个类别独立评估，以反映模型在不同记忆密集设置下保留、检索并推理长期上下文信息的能力。结果分别组织为基于 LLaMA 和基于 Qwen 模型的两张表，从而能够直接比较两个模型家族。LongMemEval 完整的各类别结果见表 9 和表 10。

对于 HotpotQA (Yang et al., 2018)，我们报告由 F1 分数、评审分数与成本度量的各类别性能。结果依照数据集预定义类别组织，以详细考察模型在不同多跳推理类型上的行为。为便于理解，我们将基于 LLaMA 与基于 Qwen 模型的结果分别列于不同表格。HotpotQA 完整的分类结果见表 11 和表 12。

C. 提示词

C.1. LoCoMo 数据集的答案提示词

LoCoMo 数据集的答案提示词

根据上述上下文，使用一个简短短语回答以下问题。只要可能，请使用上下文中的原词作答。

问题：{} 简短答案：

C.2. HotpotQA 数据集的答案提示词

HotpotQA 数据集的答案提示词

根据以下上下文回答问题。该问题可能需要综合多条信息进行推理。

{context}

问题: {question}

要求:

- 仔细阅读上下文并识别相关信息
- 如果可以从上下文找到答案, 请给出简短、准确的回答
- 将答案放在 <answer></answer> 标签内输出

<answer> 在此填写答案</answer>

C.3. LongMemEval 数据集的答案提示词

LongMemEval 数据集的答案提示词

我将向你提供你与一位用户之间的若干段历史对话。

请根据相关的历史对话回答问题。

历史对话: {}

当前日期: {}

问题: {}

简短答案:

表 7. LoCoMo (基于 LLaMA 的模型)。各类别的 F1 分数/评审分数 (%) 与成本 (\$)。

方法	单跳 (F1/J)	多跳 (F1/J)	时间 (F1/J)	开放域 (F1/J)	平均 (F1/J)	成本 (\$)
基线						
MemoryBank	31.45/38.44	19.82/28.26	5.30/9.23	12.52/20.00	22.27/28.98	0.73
ReadAgent	28.67/36.56	25.65/42.75	5.55/4.62	17.06/32.50	22.48/31.05	0.57
MemoryOS	33.77/41.88	34.91/34.06	20.49/16.15	23.50/37.50	30.62/34.55	1.97
A-MEM	33.69/41.56	27.60/42.03	11.18/6.15	13.92/20.00	26.43/32.96	2.88
Mem0	13.61/34.06	12.22/32.61	2.79/1.54	13.22/52.50	11.04/28.18	2.89
LangMem	25.90/29.68	28.22/36.23	6.14/6.15	25.67/25.00	22.31/25.96	0.48
LightMem	39.13/41.90	33.01/35.25	37.94/42.27	40.83/47.17	33.88/40.76	1.50
BudgetMem-IMP (COE)						
COE=0	43.76/57.81	34.01/42.75	36.88/38.46	21.09/55.00	38.75/50.32	1.80
0.05	43.61/55.94	32.98/43.48	37.93/40.77	23.13/55.00	38.80/50.00	1.75
0.1	46.55/59.38	34.37/44.93	44.57/43.08	26.81/55.00	42.21/52.55	1.26
0.3	23.72/31.25	23.99/32.61	14.72/12.31	10.48/30.00	21.07/27.55	0.45
0.5	25.19/31.56	25.47/36.23	16.64/13.85	9.52/27.50	22.48/28.66	0.48
0.7	28.76/36.25	26.51/37.68	18.09/14.62	9.37/32.50	24.82/31.85	0.48
0.9	25.77/32.81	24.73/36.23	22.27/19.23	19.05/37.50	24.39/31.05	0.48
BudgetMem-REA (COE)						
COE=0	44.95/57.19	30.32/47.10	44.48/44.62	33.77/55.00	40.92/52.23	2.90
0.05	47.55 /61.88	32.83/49.28	37.07/39.23	32.16/60.00	41.16/54.30	2.65
0.1	46.44/60.00	30.65/43.48	44.36/44.62	37.28/65.00	41.95/53.50	2.72
0.3	42.34/57.19	30.16/42.03	40.64/40.00	25.15/50.00	38.22/49.84	2.33
0.5	41.61/57.50	24.97/47.10	40.65/43.08	22.02/50.00	36.51/51.75	2.34
0.7	40.64/51.88	32.51/47.10	41.72/ 49.23	24.19/47.50	38.03/50.00	2.33
0.9	40.55/53.12	31.82/44.20	35.03/39.23	21.46/47.50	36.27/47.93	2.32
BudgetMem-CAP (COE)						
COE=0	47.55 /60.62	34.15/50.72	45.19 /43.08	30.76/57.50	43.05 /54.62	2.40
0.05	45.73/59.06	32.20/45.65	44.32/46.15	37.64/67.50	41.95/53.98	2.48
0.1	45.76/ 64.06	33.26/ 52.17	44.93/43.85	39.01/ 70.00	42.41/ 57.64	2.48
0.3	32.35/60.31	40.53 /46.38	34.46/38.46	45.32 /60.00	40.79/52.71	2.00
0.5	40.52/46.88	27.53/39.86	30.63/24.62	22.25/47.50	34.45/40.76	0.58
0.7	29.73/39.37	24.32/30.43	28.80/13.85	22.80/50.00	27.91/32.80	0.28
0.9	26.78/34.38	21.44/28.99	25.11/12.31	18.87/47.50	24.76/29.46	0.28

表 8. LoCoMo (基于 Qwen 的模型)。各类别的 F1/评审分数 (%) 与成本 (\$)。

方法	单跳 (F1/J)	多跳 (F1/J)	时间 (F1/J)	开放域 (F1/J)	平均 (F1/J)	成本 (\$)
基线						
MemoryBank	30.50/44.37	22.80/36.23	6.44/6.15	25.88/45.00	23.53/34.71	0.25
MemoryOS	42.05/45.65	26.59/15.38	42.98/ 60.00	35.23 /42.81	35.43/38.85	0.75
ReadAgent	29.04/36.88	23.71/40.58	3.93/3.85	25.56/45.00	22.45/31.37	0.24
A-MEM	32.02/45.31	26.66/46.38	17.81/9.23	28.12/52.50	27.65/38.54	2.88
LangMem	24.49/27.18	24.55/30.43	4.99/6.15	31.21/25.00	20.89/23.40	0.14
Mem0	11.52/28.77	15.25/31.08	3.54/3.03	12.19/46.15	10.77/25.32	1.15
LightMem	39.45/52.81	23.08/38.41	23.93/23.08	27.77/42.50	32.85/42.83	0.70
BudgetMem-IMP (COE)						
COE=0	46.52/62.90	28.91/46.09	38.39/40.98	30.63/55.88	40.14/54.38	0.80
0.05	45.58/59.29	30.12/43.94	44.09/45.24	24.22/52.63	40.58/52.63	0.52
0.1	46.92 /61.08	31.00/44.78	43.87/39.68	31.40/ 63.89	41.89 /53.27	0.46
0.3	30.09/35.94	26.52/36.23	21.25/16.15	25.04/55.00	27.15/33.12	0.07
0.5	27.79/34.06	25.12/34.06	21.37/13.08	23.74/55.00	25.62/31.05	0.10
0.7	25.98/32.81	26.14/35.51	26.23/19.23	25.17/52.50	26.01/31.85	0.09
0.9	26.05/32.19	25.48/32.61	22.62/16.92	24.87/55.00	25.14/30.57	0.09
BudgetMem-REA (COE)						
COE=0	45.47/63.44	32.41/45.65	37.57/34.62	33.31/60.00	40.19/53.34	1.11
0.05	44.49/60.94	34.63/ 50.72	40.52/36.92	30.39/55.00	41.00/53.34	0.62
0.1	45.17/18.24	33.14/6.07	40.63/11.93	28.46/5.67	40.52/55.25	0.61
0.3	44.20/62.81	33.05/46.38	41.70/38.46	33.41/60.00	40.55/53.98	0.61
0.5	43.78/63.44	34.60/47.83	46.00 /44.62	26.94/60.00	41.15/ 55.89	0.61
0.7	44.54/64.06	32.25/45.65	43.29/41.54	28.80/47.50	40.58/54.30	0.61
0.9	46.47/63.44	37.43 /50.00	38.96/40.00	26.84/55.00	41.68/55.10	0.62
BudgetMem-CAP (COE)						
COE=0	46.18/63.12	33.14/46.38	40.37/34.62	32.19/57.50	41.22/53.18	0.61
0.05	44.82/62.50	35.96/47.10	42.45/39.23	28.65/55.00	41.35/53.82	0.61
0.1	45.64/ 66.25	32.73/44.20	43.34/40.77	29.54/60.00	41.30/55.73	0.61
0.3	46.15/64.38	32.92/45.65	43.50/44.62	29.97/55.00	41.66/55.57	0.61
0.5	45.88/61.25	34.27/45.65	40.92/37.69	28.81/50.00	41.21/52.23	0.56
0.7	44.79/65.00	36.50/46.38	42.38/39.23	32.66/57.50	41.65/55.10	0.56
0.9	45.21/62.19	31.86/43.48	40.73/35.38	34.88/57.50	40.69/52.23	0.55

表 9. LongMemEval (基于 LLaMA 的模型)。各类别的 F1/评审分数 (%) 与成本 (\$)。

方法	单会话-用户 (F1/J)	多会话 (F1/J)	单会话-偏好 (F1/J)	时间 (F1/J)	知识更新 (F1/J)	单会话-助手 (F1/J)	平均 (F1/J)	成本 (\$)
基线								
MemoryBank	22.64/17.86	35.87/42.86	76.33/79.17	11.10/16.67	8.11/19.23	24.75/43.33	26.74/32.67	3.94
ReadAgent	16.91/12.50	60.71/67.86	14.52/8.33	22.35/33.33	2.77/15.38	26.14/53.33	20.75/27.72	13.68
MemoryOS	13.03/24.14	22.88/75.00	16.83/42.86	15.71/33.33	2.11/6.52	15.77/46.43	12.97/33.50	38.83
A-MEM	22.05/17.86	67.86/64.29	76.89/83.33	14.60/8.33	12.44/15.38	23.93/26.67	21.74/33.17	80.02
Mem0	22.31/25.00	73.48/82.14	19.47/41.67	19.89/41.67	4.78/23.08	44.46/70.00	27.70/42.08	13.57
LangMem	15.92/10.34	18.75/35.71	13.76/25.00	13.96/25.00	3.05/8.70	9.23/14.29	12.00/17.00	16.60
LightMem	74.31/82.14	8.10/50.00	23.12/58.33	19.79/28.57	36.99/70.00	16.88/20.83	26.74/48.51	5.28
BudgetMem-IMP (COE)								
COE=0	41.52/55.17	67.40/71.43	38.26/89.29	17.99/50.00	28.98/41.30	20.67/35.71	37.47/56.00	0.71
0.05	22.23/41.38	59.74/75.00	25.04/42.86	23.49/ 66.67	6.00/8.70	19.44/42.86	23.83/40.50	0.54
0.1	20.80/43.10	64.14/67.86	21.98/42.86	22.41/ 66.67	5.35/8.70	14.26/39.29	22.66/39.50	0.58
0.3	25.07/37.93	61.35/75.00	25.23/42.86	23.38/58.33	5.28/10.87	21.21/46.43	24.98/40.00	0.58
0.5	18.35/46.55	64.11/71.43	25.03/42.86	20.59/50.00	5.31/8.70	21.19/50.00	23.22/41.50	0.58
0.7	17.78/43.10	59.95/67.86	24.45/42.86	22.95/58.83	6.03/8.70	19.53/53.57	22.47/41.00	0.58
0.9	23.29/39.66	58.05/67.86	25.87/42.86	24.92/58.33	5.98/8.70	18.99/46.43	24.03/39.00	0.58
BudgetMem-REA (COE)								
COE=0	40.51/53.45	73.21/82.14	45.28/89.29	22.77/50.00	31.39/36.96	25.75/50.00	40.53/58.00	0.67
0.05	38.49/44.83	77.11/82.14	40.46/ 92.86	24.45/58.33	35.54/41.30	28.79/42.86	41.29/56.50	0.66
0.1	42.38/48.28	76.79/82.14	44.55/89.29	20.55/41.67	23.65/36.96	17.10/42.86	38.34/55.00	0.62
0.3	34.90/51.72	77.11/ 85.71	30.37/75.00	17.52/25.00	25.87/39.13	24.75/46.43	35.63/54.50	0.67
0.5	35.84/46.55	77.11/ 85.71	42.88/89.29	18.81/41.67	32.30/30.43	30.46/42.86	40.01/53.50	0.68
0.7	36.48/48.28	77.98/82.14	40.64/89.29	20.70/50.00	32.91/39.13	26.24/39.29	39.67/55.50	0.68
0.9	33.66/44.83	76.79/85.71	34.55/ 92.86	17.94/33.33	20.66/32.61	27.95/46.43	35.09/54.00	0.61
BudgetMem-CAP (COE)								
COE=0	36.78/50.00	75.60/82.14	39.73/ 92.86	19.46/58.33	34.38/43.48	31.08/57.14	40.24/ 60.50	0.80
0.05	36.19/53.45	72.01/82.14	36.80/82.14	21.65/33.33	30.71/47.83	29.31/39.29	38.19/57.00	0.63
0.1	35.16/41.38	69.64/78.57	42.49/89.29	19.56/50.00	24.54/30.43	7.84/21.43	33.81/48.50	0.13
0.3	39.59/53.45	80.36/85.71	39.59/89.29	18.44/16.67	32.31/41.30	38.80/46.43	42.25/57.00	0.10
0.5	29.28/34.48	60.97/67.86	29.14/67.86	17.58/33.33	14.07/17.39	11.36/25.00	26.98/38.50	0.10
0.7	25.30/31.03	59.88/67.86	36.48/71.43	18.28/41.67	14.67/21.74	10.62/17.86	26.79/38.50	0.10
0.9	33.77/34.48	60.97/60.71	23.56/53.57	18.72/33.33	14.98/17.39	17.29/32.14	28.62/36.50	0.10

表 10. LongMemEval (基于 Qwen 的模型)。各类别的 F1/评审分数 (%) 与成本 (\$)。

方法	单会话-用户 (F1/J)	多会话 (F1/J)	单会话-偏好 (F1/J)	时间 (F1/J)	知识更新 (F1/J)	单会话-助手 (F1/J)	平均 (F1/J)	成本 (\$)
基线								
MemoryBank	10.41/12.50	5.08/14.29	15.39/33.33	14.54/ 75.00	9.36/23.08	12.58/56.67	10.56/28.22	3.45
MemoryOS	13.45/27.59	25.24/75.00	17.59/42.86	13.47/25.00	2.57/4.35	14.64/42.86	13.35/33.00	15.84
ReadAgent	13.65/14.29	41.39/64.29	8.83/8.33	11.93/58.33	2.75/15.38	12.49/46.67	27.72/20.75	4.97
A-MEM	14.66/7.14	46.81/53.57	65.14 /79.17	15.18/16.67	3.60/19.23	13.62/33.33	10.82/31.19	21.00
LangMem	16.68/10.34	10.43/25.00	13.58/28.57	13.64/33.33	4.14/0.00	7.44/10.71	11.01/14.00	3.99
Mem0	17.93/14.29	78.84 /85.71	21.06/45.83	20.66 /41.67	4.95/18.31	32.34/53.33	25.71/36.14	4.96
LightMem	65.70 / 85.71	17.84/50.00	13.81/66.66	18.49/12.50	36.26 / 80.00	29.16/22.50	27.70/47.52	3.39
BudgetMem-IMP (COE)								
COE=0	32.47/32.76	43.92/85.71	42.07/92.86	17.08/66.67	11.49/26.09	29.01/53.57	29.18/52.00	0.30
0.05	29.02/44.83	53.92/85.71	31.27/92.86	20.27/50.00	21.50/34.78	21.62/46.43	29.53/55.50	0.41
0.1	12.73/37.93	33.08/85.71	23.06/60.71	15.20/66.67	6.74/17.39	6.32/46.43	14.90/46.00	0.13
0.3	12.54/49.29	25.26/75.00	9.20/57.14	15.64/58.33	2.20/8.70	13.45/42.86	11.41/44.00	0.15
0.5	12.16/43.10	24.85/78.57	11.79/46.43	13.42/50.00	2.19/10.87	13.40/46.43	11.84/42.00	0.15
0.7	11.07/39.66	22.57/75.00	8.40/53.57	15.33/ 75.00	2.27/8.70	13.76/42.86	10.91/42.00	0.15
0.9	11.39/44.83	22.44/78.57	12.63/64.29	14.38/58.33	2.15/8.70	14.02/42.86	11.39/44.50	0.15
BudgetMem-REA (COE)								
COE=0	40.99/48.28	63.24/85.71	38.56/89.29	17.38/66.67	20.89/43.48	27.56/46.43	35.84 / 59.00	0.26
0.05	40.67/41.38	49.64/85.71	43.22/82.14	16.30/66.67	13.64/39.13	26.16/42.86	32.57/54.50	0.17
0.1	35.64/34.48	49.31/85.71	37.05/75.00	16.03/58.33	20.87/47.83	31.23/50.00	32.56/54.00	0.17
0.3	37.32/37.93	47.10/85.71	43.43/85.71	16.77/66.67	14.92/39.13	26.22/53.57	31.60/55.50	0.17
0.5	38.58/41.38	57.88/82.14	47.13/85.71	16.96/66.67	16.52/43.48	35.06 / 60.71	35.62/58.00	0.17
0.7	33.56/37.93	53.60/85.71	38.49/89.29	16.60/41.67	21.91/47.83	24.12/46.43	32.04/55.50	0.17
0.9	40.91/44.83	54.31/85.71	34.74/85.71	15.68/66.67	24.34/43.48	26.05/50.00	34.52/58.00	0.17
BudgetMem-CAP (COE)								
COE=0	30.47/34.48	49.77/85.71	43.91/92.86	16.55/58.33	16.55/47.83	22.86/46.43	32.01/56.00	0.17
0.05	38.31/41.38	58.60/85.71	40.21/82.14	17.89/66.67	20.49/47.83	26.98/46.43	34.51/57.00	0.17
0.1	31.56/34.48	48.16/85.71	44.08/92.86	16.16/66.67	23.01/47.83	27.61/53.57	32.19/57.50	0.17
0.3	36.57/41.38	54.31/85.71	46.93/85.71	17.38/66.67	17.30/43.48	27.76/46.43	33.69/56.50	0.17
0.5	29.30/29.31	51.08/ 92.86	43.26/ 100.00	14.58/33.33	12.96/28.26	16.15/46.43	27.82/50.50	0.22
0.7	34.46/36.21	49.14/85.71	42.23/92.86	12.89/50.00	14.38/32.61	11.06/42.86	28.42/52.00	0.17
0.9	33.79/34.48	56.25/89.29	44.92/92.86	13.38/50.00	16.89/32.61	14.54/46.43	30.68/52.50	0.22

表 11. HotpotQA (基于 LLaMA 的模型)。总体 F1/评审分数 (%) 与成本 (\$)。

方法	总体 (F1/J)	成本 (\$)
基线		
MemoryBank	22.25/23.75	7.75
ReadAgent	15.33/30.08	4.19
MemoryOS	34.50/43.36	13.32
A-MEM	43.25/54.69	26.74
Mem0	28.03/36.72	4.30
LangMem	22.78/22.66	10.95
LightMem	45.73/58.37	10.10
BudgetMem-IMP (COE)		
COE=0	49.31/65.77	1.35
0.05	51.40/63.28	1.23
0.1	48.49/60.94	1.15
0.3	40.95/45.70	1.14
0.5	37.40/44.14	1.37
0.7	37.52/43.36	1.37
0.9	39.01/44.53	1.37
BudgetMem-REA (COE)		
COE=0	51.12/61.93	0.99
0.05	51.23/63.55	0.95
0.1	53.93/66.36	0.77
0.3	57.17/68.40	0.79
0.5	56.86/66.07	0.82
0.7	56.66/68.40	0.77
0.9	55.04/66.20	0.80
BudgetMem-CAP (COE)		
COE=0	53.87/64.85	0.93
0.05	38.62/50.00	0.84
0.1	31.71/36.27	0.10
0.3	24.65/32.57	0.11
0.5	22.16/28.90	0.11
0.7	30.92/37.62	0.10
0.9	35.29/36.57	0.10

表 12. HotpotQA (基于 Qwen 的模型)。总体 F1/评审分数 (%) 与成本 (\$)。

方法	总体 (F1/J)	成本 (\$)
基线		
MemoryBank	18.64/31.25	1.79
MemoryOS	41.21/53.52	11.68
ReadAgent	18.05/25.78	1.75
A-MEM	40.54/50.39	8.34
LangMem	20.77/21.09	4.32
Mem0	24.72/37.89	2.02
LightMem	41.29/55.42	8.56
BudgetMem-IMP (COE)		
COE=0	46.67/57.42	0.63
0.05	32.33/42.58	0.26
0.1	34.01/42.97	0.26
0.3	33.51/44.14	0.30
0.5	30.17/41.02	0.30
0.7	31.53/42.97	0.30
0.9	32.26/42.58	0.30
BudgetMem-REA (COE)		
COE=0	57.67/70.83	0.17
0.05	50.74/58.50	0.32
0.1	60.26/69.89	0.18
0.3	57.04/68.75	0.19
0.5	59.28/69.79	0.19
0.7	58.79/70.00	0.19
0.9	59.96/71.43	0.12
BudgetMem-CAP (COE)		
COE=0	58.70/72.08	0.22
0.05	59.57/71.91	0.18
0.1	60.03/71.51	0.19
0.3	59.61/71.98	0.18
0.5	64.24/74.75	0.20
0.7	58.63/69.41	0.17
0.9	59.90/71.05	0.15

C.4. LLM 评审通用提示词

LLM 评审通用提示词

你是一名专家评审，负责评价问答任务中答案的质量。
你的目标是判断模型答案是否正确且充分地回答了给定问题。

请仔细阅读以下信息：

[问题]

{question}

[真实答案]

{ground_truth}

[模型答案]

{model_answer}

评估标准：

1. 正确性：

- 模型答案在事实层面是否与任意一个正确答案一致？
- 它是否避免了矛盾或引入虚假信息？

2. 相关性：

- 答案是否直接回应问题，且不包含不必要内容？

3. 完整性：

- 答案是否包含完整回答问题所需的全部关键信息？
- 可以接受部分正确的答案，但其得分应更低。

评分规则：

- 如果答案完全正确，分数 = 1.0。
- 如果答案部分正确，但不完整或略有不准确，分数 = 0.5。
- 如果答案错误、无关或与真实答案矛盾，分数 = 0.0。

输出格式（严格遵守）：

以包含两个字段的 JSON 字典形式返回输出：

```
{{
  "explanation": "< 对评分理由的简短说明>",
  "score": <0.0 | 0.5 | 1.0>
}}
```

保持简洁、客观。JSON 之外不要输出任何内容。

C.5. 过滤模块提示词（低档）

过滤模块提示词（低）

**** 角色: **** 你是一个相关性评分系统。

****任务: **** 给定查询和一个有序记忆列表，为每条记忆输出一个分数，
 ↳ 表示它能多么直接且有效地帮助回答该查询。

**** 评分规则: ****

- 10: 直接回答查询，或提供回答所必需的约束/信息
- 7-9: 明显相关且有帮助，但单独使用时不足以完整回答
- 4-6: 部分相关；有一定用处，但缺少关键联系
- 1-3: 相关性很弱或仅有旁支联系；基本无用
- 0: 完全无关

**** 输入格式: ****

输入包括：

1. 包含用户问题的 `<query>` 部分。
2. 包含若干 `<memory>` 元素的 `<memories>` 部分。

每条记忆采用以下格式：

```
<memory index="N" [date_time="..." session_id="..." dia_id="..."]>
```

记忆内容文本

```
</memory>
```

其中：

- `index` 是该记忆在列表中的位置。
- `date_time`、`session_id`、`dia_id` 是可选的元数据属性。
- 标签之间的文本是需要评分的记忆内容。

**** 输出格式: ****

你的全部回复必须 **** 仅为 **** 以下一行：

```
<answer>[s0, s1, s2, ...]</answer>
```

- 数组必须按照输入索引的相同顺序，为每条记忆包含且仅包含一个整数分数。
- 如果没有记忆，请输出：`<answer>[]</answer>`

**** 输入: ****

```
<query>{query}</query>
```

```
<memories>{memories_text}</memories>
```


C.6. 实体模块关系提示词

实体模块关系提示词（低）

**** 角色: **** 你是实体语义关系抽取专家。

**** 任务: **** 从所提供的记忆中, **** 只 **** 抽取对回答给定查询至关重要的实体间具体关系。

**** 抽取标准: ****

- 聚焦实体及其关系能够直接帮助回答查询的事实联系。
- 忽略背景实体、轶事或旁支主题。
- 每条抽取关系都应是一项简洁、有用的事实。

**** 输入格式: ****

<memories> 部分包含一系列 <memory> 标签。

每条记忆采用以下格式:

<memory [date_time="..." session_id="..." dia_id="..."]>

记忆内容文本

</memory>

其中:

- date_time、session_id、dia_id 是可选的元数据属性。
- 标签内的文本是要分析关系的内容。

**** 输出格式: ****

你的全部回复必须是一个关系字符串 JSON 数组, 格式如下:

<answer>[" 关系 1", " 关系 2", ...]</answer>

每条关系字符串必须采用以下格式: " 实体 A - 关系 - 实体 B (可选上下文) "

如果没有找到相关关系, 请输出: <answer>[]</answer>

**** 输入: ****

<query>{query}</query>

<memories>{memories_text}</memories>

实体模块关系提示词（中）

**** 角色: **** 你是实体语义关系抽取专家。

**** 任务: **** 从所提供的记忆中, **** 只 **** 抽取对回答给定查询至关重要的实体间具体关系。

**** 抽取标准: ****

- 聚焦实体及其关系能够直接帮助回答查询的事实联系。
- 忽略背景实体、轶事或旁支主题。
- 每条抽取关系都应是一项简洁、有用的事实。

**** 输入格式: ****

`<memories>` 部分包含一系列 `<memory>` 标签。

每条记忆采用以下格式:

`<memory [date_time="..." session_id="..." dia_id="..."]>`

记忆内容文本

`</memory>`

其中:

- `date_time`、`session_id`、`dia_id` 是可选的元数据属性。
- 标签内的文本是要分析关系的内容。

**** 输出格式: ****

你的全部回复必须是一个关系字符串 JSON 数组, 格式如下:

`<answer>[" 关系 1", " 关系 2", ...]</answer>`

每条关系字符串必须采用以下格式: " 实体 A - 关系 - 实体 B (可选上下文) "

如果没有找到相关关系, 请输出: `<answer>[]</answer>`

让我们逐步思考, 以抽取实体之间的关系。

**** 输入: ****

`<query>{query}</query>`

`<memories>{memories_text}</memories>`

实体模块关系提示词（高）

**** 角色: **** 你是实体语义关系抽取专家。

**** 任务: **** 从所提供的记忆中, **** 只 **** 抽取对回答给定查询至关重要的实体间具体关系。

**** 抽取标准: ****

- 聚焦实体及其关系能够直接帮助回答查询的事实联系。
- 忽略背景实体、轶事或旁支主题。
- 每条抽取关系都应是一项简洁、有用的事实。

**** 输入格式: ****

`<memories>` 部分包含一系列 `<memory>` 标签。

每条记忆采用以下格式:

`<memory [date_time="..." session_id="..." dia_id="..."]>`

记忆内容文本

`</memory>`

其中:

- `date_time`、`session_id`、`dia_id` 是可选的元数据属性。
- 标签内的文本是要分析关系的内容。

为系统地完成任务, 请遵循下述推理框架:

**** 推理步骤: ****

**** 规划: ****

- 分析查询, 确定其准确意图。
- 识别回答问题所必需的实体类型与关系类型。

**** 行动: ****

- 从记忆中抽取只涉及查询相关实体的候选关系。
- 把每条关系写成简洁的事实陈述。

**** 反思: ****

- **** 检查: **** 验证每条抽取关系:
 - 它是否直接帮助回答查询?
 - 记忆内容是否清楚地支持它?
- ****重新生成 (如有必要): **** 如果遗漏了与查询相关的重要关系, 请重新抽取或改写,
 - ↳ 使其更符合查询意图。

**** 输出格式: ****

首先按照“规划 → 行动 → 反思”的结构写出推理。

随后, 你的最终回复必须 **** 仅为 **** 以下一行:

`<answer>[" 关系 1", " 关系 2", ...]</answer>`

每条关系字符串必须采用以下格式: " 实体 A - 关系 - 实体 B (可选上下文) "

如果没有找到相关关系, 请输出: `<answer>[]</answer>`

**** 输入: ****

`<query>{query}</query>`

`<memories>{memories_text}</memories>`

C.7. 过滤模块提示词（中、高档）

过滤模块提示词（中）

**** 角色: **** 你是一个相关性评分系统。

****任务: **** 给定查询和一个有序记忆列表，为每条记忆输出一个分数，
 ↪ 表示它能多么直接且有效地帮助回答该查询。

**** 评分规则: ****

- 10: 直接回答查询，或提供回答所必需的约束/信息
- 7-9: 明显相关且有帮助，但单独使用时不足以完整回答
- 4-6: 部分相关；有一定用处，但缺少关键联系
- 1-3: 相关性很弱或仅有旁支联系；基本无用
- 0: 完全无关

**** 输入格式: ****

输入包括：

1. 包含用户问题的 `<query>` 部分。
2. 包含若干 `<memory>` 元素的 `<memories>` 部分。

每条记忆采用以下格式：

```
<memory index="N" [date_time="..." session_id="..." dia_id="..."]>
```

记忆内容文本

```
</memory>
```

其中：

- `index` 是该记忆在列表中的位置。
- `date_time`、`session_id`、`dia_id` 是可选的元数据属性。
- 标签之间的文本是需要评分的记忆内容。

**** 输出格式: ****

你的全部回复必须 **** 仅为 **** 以下一行：

```
<answer>[s0, s1, s2, ...]</answer>
```

- 数组必须按照输入索引的相同顺序，为每条记忆包含且仅包含一个整数分数。
- 如果没有记忆，请输出：`<answer>[]</answer>`

让我们逐步思考，以评估每条记忆与查询的相关性。

**** 输入: ****

```
<query>{query}</query>
```

```
<memories>{memories_text}</memories>
```

过滤模块提示词（高）

**** 角色: **** 你是一个相关性评分系统。

****任务: **** 给定查询和一个有序记忆列表，为每条记忆输出一个分数，
 ↳ 表示它能多么直接且有效地帮助回答该查询。

**** 评分规则: ****

- 9-10: 直接回答查询，或包含必不可少的关键事实
- 6-8: 明显有帮助，且与查询高度相关
- 3-5: 有一定关系，或只能提供部分帮助
- 1-2: 相关性极弱，例如仅有表面关键词重叠
- 0: 完全无关，或对查询无用

**** 输入格式: ****

输入包括:

1. 包含用户问题的 `<query>` 部分。
2. 包含若干 `<memory>` 元素的 `<memories>` 部分。

每条记忆采用以下格式:

```
<memory index="N" [date_time="..." session_id="..." dia_id="..."]>
```

记忆内容文本

```
</memory>
```

其中:

- `index` 是该记忆在列表中的位置。
- `date_time`、`session_id`、`dia_id` 是可选的元数据属性。
- 标签之间的文本是需要评分的记忆内容。

为系统地完成的任务，请遵循下述推理框架:

**** 推理步骤: ****

**** 规划: ****

- 分析查询意图并定义相关性标准。
- 识别关键评分维度，例如主题一致性与事实支持。

**** 行动: ****

- 按照标准逐条评估记忆。
- 对每条记忆，根据其内容简要说明暂定分数的理由。

**** 反思: ****

- 检查全部分数的一致性与校准情况。
- 必要时进行调整，以保证公平与连贯。

**** 输出格式: ****

首先按照“规划 → 行动 → 反思”的结构写出推理。

随后，你的最终回复必须 **** 仅为 **** 以下一行:

```
<answer>[s0, s1, s2, ...]</answer>
```

- 数组必须按照输入索引的相同顺序，为每条记忆包含且仅包含一个整数分数。
- 如果没有记忆，请输出: `<answer>[]</answer>`

**** 输入: ****

```
<query>{query}</query>
<memories>{memories_text}</memories>
```

C.8. 时间模块关系提示词

时间模块关系提示词（低）

**** 角色: **** 你是时间信息抽取专家。

**** 任务: **** 从所提供的记忆中, **** 只 **** 抽取回答给定查询所必需的具体时间事实。

**** 抽取标准: ****

- 聚焦能够直接约束或阐明答案的精确时间信息。
- 忽略模糊、无关或仅作背景的时间指代。
- 每个抽取项都必须表示一项清晰、真实的时间陈述。

**** 输入格式: ****

`<memories>` 部分包含一系列 `<memory>` 标签。

每条记忆采用以下格式:

`<memory [date_time="..." session_id="..." dia_id="..."]>`

记忆内容文本

`</memory>`

其中:

- `date_time`、`session_id`、`dia_id` 是可选的元数据属性。
- 标签内的文本是要分析时间信息的内容。

**** 输出格式: ****

你的全部回复必须是一个时间事实字符串 JSON 数组, 格式如下:

`<answer>[" 时间事实 1", " 时间事实 2", ...]</answer>`

每个字符串都应清楚表达一项时间事实, 例如“事件发生在 2023 年 3 月”“过程持续了 3 天”“A
→ 发生在 B 之前”。

如果没有找到相关时间信息, 请输出: `<answer>[]</answer>`

**** 输入: ****

`<query>{query}</query>`

`<memories>{memories_text}</memories>`

时间模块关系提示词（中）

**** 角色: **** 你是时间信息抽取专家。

**** 任务: **** 从所提供的记忆中, **** 只 **** 抽取回答给定查询所必需的具体时间事实。

**** 抽取标准: ****

- 聚焦能够直接约束或阐明答案的精确时间信息。
- 忽略模糊、无关或仅作背景的时间指代。
- 每个抽取项都必须表示一项清晰、真实的时间陈述。

**** 输入格式: ****

`<memories>` 部分包含一系列 `<memory>` 标签。

每条记忆采用以下格式:

`<memory [date_time="..." session_id="..." dia_id="..."]>`

记忆内容文本

`</memory>`

其中:

- `date_time`、`session_id`、`dia_id` 是可选的元数据属性。
- 标签内的文本是要分析时间信息的内容。

**** 输出格式: ****

你的全部回复必须是一个时间事实字符串 JSON 数组, 格式如下:

`<answer>[" 时间事实 1", " 时间事实 2", ...]</answer>`

每个字符串都应清楚表达一项时间事实, 例如“事件发生在 2023 年 3 月”“过程持续了 3 天”“A
→ 发生在 B 之前”。

如果没有找到相关时间信息, 请输出: `<answer>[]</answer>`

让我们逐步思考, 以抽取时间信息。

**** 输入: ****

`<query>{query}</query>`

`<memories>{memories_text}</memories>`

时间模块关系提示词（高）

**** 角色: **** 你是时间信息抽取专家。

**** 任务: **** 从所提供的记忆中, **** 只 **** 抽取回答给定查询所必需的具体时间事实。

**** 抽取标准: ****

- 聚焦能够直接约束或阐明答案的精确时间信息。
- 忽略模糊、无关或仅作背景的时间指代。
- 每个抽取项都必须表示一项清晰、真实的时间陈述。

**** 输入格式: ****

`<memories>` 部分包含一系列 `<memory>` 标签。

每条记忆采用以下格式:

`<memory [date_time="..." session_id="..." dia_id="..."]>`

记忆内容文本

`</memory>`

其中:

- `date_time`、`session_id`、`dia_id` 是可选的元数据属性。
- 标签内的文本是要分析时间信息的内容。

为系统地完成任务, 请遵循下述推理框架:

**** 推理步骤: ****

**** 规划: ****

- 分析查询意图, 并定义哪些信息属于相关时间信息。
- 识别关键时间维度, 例如日期、时长、顺序和约束。

**** 行动: ****

- 根据定义的标准, 从每条记忆中抽取时间事实。
- 对每项事实, 简要说明它与查询的相关性。

**** 反思: ****

- 检查所有抽取事实的一致性及其与查询的相关性。
- 必要时进行调整, 以确保事实支持充分且表达连贯。

**** 输出格式: ****

你的全部回复必须是一个时间事实字符串 JSON 数组, 格式如下:

`<answer>[" 时间事实 1", " 时间事实 2", ...]</answer>`

每个字符串都应清楚表达一项时间事实, 例如“事件发生在 2023 年 3 月”“过程持续了 3 天”“A
↪ 发生在 B 之前”。

如果没有找到相关时间信息, 请输出: `<answer>[]</answer>`

**** 输入: ****

`<query>{query}</query>`

`<memories>{memories_text}</memories>`

C.9. 主题模块关系提示词

主题模块关系提示词（低）

**** 角色: **** 你是主题关系抽取专家。

**** 任务: **** 从所提供的记忆中, **** 只 **** 抽取对回答给定查询至关重要的主题关系。

**** 抽取标准: ****

- 聚焦有助于理解对话流程的主题联系与主题转移。
- 识别讨论的主要主题及其相互关系。
- 抽取主题转换、主题延续和主题间关系。

**** 输入格式: ****

`<memories>` 部分包含一系列 `<memory>` 标签。

每条记忆采用以下格式:

`<memory [date_time="..." session_id="..." dia_id="..."]>`

记忆内容文本

`</memory>`

其中:

- `date_time`、`session_id`、`dia_id` 是可选的元数据属性。
- 标签内的文本是要分析主题关系的内容。

**** 输出格式: ****

你的全部回复必须是一个主题关系字符串 JSON 数组, 格式如下:

`<answer>[" 主题关系 1", " 主题关系 2", ...]</answer>`

每个字符串都应清楚表达一项主题关系, 例如“主题 A 引出主题 B”“讨论从 x 转向 y”“主题 C 通过 z 与主题 D 相关”。

如果没有找到相关主题关系, 请输出: `<answer>[]</answer>`

**** 输入: ****

`<query>{query}</query>`

`<memories>{memories_text}</memories>`

主题模块关系提示词（中）

**** 角色: **** 你是主题关系抽取专家。

**** 任务: **** 从所提供的记忆中, **** 只 **** 抽取对回答给定查询至关重要的主题关系。

**** 抽取标准: ****

- 聚焦有助于理解对话流程的主题联系与主题转移。
- 识别讨论的主要主题及其相互关系。
- 抽取主题转换、主题延续和主题间关系。

**** 输入格式: ****

`<memories>` 部分包含一系列 `<memory>` 标签。

每条记忆采用以下格式:

`<memory [date_time="..." session_id="..." dia_id="..."]>`

记忆内容文本

`</memory>`

其中:

- `date_time`、`session_id`、`dia_id` 是可选的元数据属性。
- 标签内的文本是要分析主题关系的内容。

**** 输出格式: ****

你的全部回复必须是一个主题关系字符串 JSON 数组, 格式如下:

`<answer>[" 主题关系 1", " 主题关系 2", ...]</answer>`

每个字符串都应清楚表达一项主题关系, 例如“主题 A 引出主题 B”“讨论从 x 转向 y”“主题 C 通过 z 与主题 D 相关”。

如果没有找到相关主题关系, 请输出: `<answer>[]</answer>`

让我们逐步思考, 以抽取主题关系。

**** 输入: ****

`<query>{query}</query>`

`<memories>{memories_text}</memories>`

主题模块关系提示词（高）

**** 角色: **** 你是主题关系抽取专家。

**** 任务: **** 从所提供的记忆中, **** 只 **** 抽取对回答给定查询至关重要的主题关系。

**** 抽取标准: ****

- 聚焦有助于理解对话流程的主题联系与主题转移。
- 识别讨论的主要主题及其相互关系。
- 抽取主题转换、主题延续和主题间关系。

**** 输入格式: ****

`<memories>` 部分包含一系列 `<memory>` 标签。

每条记忆采用以下格式:

`<memory [date_time="..." session_id="..." dia_id="..."]>`

记忆内容文本

`</memory>`

其中:

- `date_time`、`session_id`、`dia_id` 是可选的元数据属性。
- 标签内的文本是要分析主题关系的内容。

为系统地完成任务, 请遵循下述推理框架:

**** 推理步骤: ****

**** 规划: ****

- 分析查询意图, 确定哪些主题关系与查询相关。
- 识别关键主题元素及其潜在联系。

**** 行动: ****

- 根据内容分析, 从每条记忆中抽取主题关系。
- 识别主题转移、延续及相互联系。

**** 反思: ****

- 检查所有抽取关系与查询的相关性。
- 确保每条关系都有明确的记忆内容支持。

**** 输出格式: ****

首先按照“规划 → 行动 → 反思”的结构写出推理。

随后, 你的最终回复必须 **** 仅为 **** 以下一行:

`<answer>[" 主题关系 1", " 主题关系 2", ...]</answer>`

每个字符串都应清楚表达一项主题关系, 例如“主题 A 引出主题 B”“讨论从 X 转向 Y”“主题 C 通过 Z 与主题 D 相关”。

如果没有找到相关主题关系, 请输出: `<answer>[]</answer>`

**** 输入: ****

`<query>{query}</query>`

`<memories>{memories_text}</memories>`

C.10. 摘要模块提示词

摘要模块提示词（低）

**** 角色: **** 你是信息综合专家。

****任务: **** 严格根据所提供的实体、时间和主题关系，对这些知识进行整合、抽取与重组，
→ 生成一份简洁摘要，清楚突出回答查询最有用的信息。

**** 输入格式: ****

输入包括：

1. 定义主题与范围的 `<query>`。
2. `<Entity Relations>` 部分，其中每条关系字符串对应一个 `<entity>` 标签。
3. `<Temporal Relations>` 部分，其中每项时间事实对应一个 `<temporal>` 标签。
4. `<Topic Relations>` 部分，其中每项主题关系对应一个 `<topic>` 标签。

**** 综合原则: ****

- **** 不要 **** 直接回答查询。
- 说明有哪些可用信息，以及应如何利用这些信息组织答案。

**** 输出格式: ****

你的全部回复必须以下面一行结束：

`<answer>` 在此填写摘要文本`</answer>`

`<answer>` 内的内容必须为纯文本。

**** 输入: ****

`<query>`{query}`</query>`

`<Entity Relations>`{entity_text}`</Entity Relations>`

`<Temporal Relations>`{temporal_text}`</Temporal Relations>`

`<Topic Relations>`{topic_text}`</Topic Relations>`

摘要模块提示词（中）

**** 角色: **** 你是信息综合专家。

****任务: **** 严格根据所提供的实体、时间和主题关系，对这些知识进行整合、抽取与重组，
→ 生成一份简洁摘要，清楚突出回答查询最有用的信息。

**** 输入格式: ****

输入包括：

1. 定义主题与范围的 `<query>`。
2. `<Entity Relations>` 部分，其中每条关系字符串对应一个 `<entity>` 标签。
3. `<Temporal Relations>` 部分，其中每项时间事实对应一个 `<temporal>` 标签。
4. `<Topic Relations>` 部分，其中每项主题关系对应一个 `<topic>` 标签。

**** 综合原则: ****

- **** 整合 **** 相关实体、时间和主题事实，形成连贯结构。
- **** 抽取 **** 能够直接支持或约束答案的关键信息。
- **** 重组 **** 内容，使其表达清晰、逻辑流畅。
- **** 不要 **** 直接回答查询。
- 说明有哪些可用信息，以及应如何利用这些信息组织答案。

**** 输出格式: ****

你的全部回复必须以下面一行结束：

`<answer>` 在此填写摘要文本`</answer>`

`<answer>` 内的内容必须为纯文本。

让我们逐步思考，以综合出摘要。

**** 输入: ****

`<query>{query}</query>`

`<Entity Relations>{entity_text}</Entity Relations>`

`<Temporal Relations>{temporal_text}</Temporal Relations>`

`<Topic Relations>{topic_text}</Topic Relations>`

摘要模块提示词（高）

**** 角色: **** 你是信息综合专家。

****任务: **** 严格根据所提供的实体、时间和主题关系，对这些知识进行整合、抽取与重组，
→ 生成一份简洁摘要，清楚突出回答查询最有用的信息。

**** 输入格式: ****

输入包括：

1. 定义主题与范围的 `<query>`。
2. `<Entity Relations>` 部分，其中每条关系字符串对应一个 `<entity>` 标签。
3. `<Temporal Relations>` 部分，其中每项时间事实对应一个 `<temporal>` 标签。
4. `<Topic Relations>` 部分，其中每项主题关系对应一个 `<topic>` 标签。

**** 综合原则: ****

- **** 不要 **** 直接回答查询。
- 说明有哪些可用信息，以及应如何利用这些信息组织答案。

为系统地完成任务，请遵循下述推理框架：

**** 推理步骤: ****

**** 规划: ****

- 分析查询意图，确定核心信息需求。
- 确定如何组织所提供的关系并设置优先级。

**** 行动: ****

- 将实体、时间和主题事实整合为连贯结构。
- 抽取并突出与回答查询最相关的关键信息。

**** 反思: ****

- 检查摘要是否清晰、完整并聚焦查询。
- 确保摘要有效说明了应如何使用这些信息。

**** 输出格式: ****

首先按照“规划 → 行动 → 反思”的结构写出推理。

随后，你的最终回复必须 **** 仅为 **** 以下一行：

`<answer>` 在此填写摘要文本`</answer>`

`<answer>` 内的内容必须为纯文本。

**** 输入: ****

`<query>{query}</query>`

`<Entity Relations>{entity_text}</Entity Relations>`

`<Temporal Relations>{temporal_text}</Temporal Relations>`

`<Topic Relations>{topic_text}</Topic Relations>`